

THUNDERS-Master:
**un proceso ligero para construir, monitorear
y validar el entendimiento compartido en las
actividades de elicitación, especificación y
validación de requisitos de software**

Trabajo de grado presentado como requisito parcial
para optar al título de Ingeniero de Sistemas

Autores

Andrés Felipe Collazos Fernández
José David Chilito Cometa

Director

Vanessa Agredo-Delgado, Ph.D.

Codirector

Cesar Alberto Collazos, Ph.D.

Codirector

Leandro Antonelli, Ph.D.
Línea de investigación: Ingeniería de Software

Popayán, Colombia
2026

Dedicatoria

☐ Sección pendiente

Escribir la dedicatoria

Agradecimientos

☐ Sección pendiente

Redactar los agradecimientos

Resumen

El entendimiento compartido es una condición crítica para la calidad de los requisitos de software, porque permite que stakeholders, usuarios, analistas y equipos técnicos converjan en una interpretación común del problema, las necesidades, los requisitos y los criterios de validación. Sin embargo, la literatura aborda este fenómeno de forma dispersa, bajo términos como comunicación, coordinación, alineación, negociación, entendimiento común y modelos mentales compartidos, lo que dificulta usarlo como guía operativa para la elicitación, especificación y validación de requisitos. Este trabajo de grado sintetiza una revisión sistemática de la literatura y una caracterización de dichas actividades con base en la norma ISO/IEC/IEEE 29148:2018, y a partir de allí adapta el proceso THUNDERS hacia *THUNDERS-Master*, una versión ligera orientada a construir, monitorear y validar el entendimiento compartido en ingeniería de requisitos. La adaptación se realiza mediante Ingeniería de Métodos Situacionales (SME) orientada por *method chunks*, complementada con criterios de *tailoring* contextual y reducción de carga cognitiva. THUNDERS-Master conserva la lógica de tres fases de THUNDERS —Beginning, Developing y Measuring— pero la traduce a un proceso mínimo viable de once actividades operativas (A1–A11), con roles simplificados, artefactos mínimos, evidencias observables, criterios de activación contextual y trazabilidad explícita con el proceso original. El resultado se presenta como una especificación formal de trabajo, lista para revisión experta, walkthrough metodológico y piloto posterior; aún no se presenta como un proceso empíricamente validado.

Palabras clave: Ingeniería de requisitos, Entendimiento compartido, Ingeniería de Métodos Situacionales, Adaptación de procesos, THUNDERS, THUNDERS-Master.

Abstract

Shared understanding is a critical condition for software requirements quality, because it allows stakeholders, users, analysts, and technical teams to converge on a common interpretation of the problem, the needs, the requirements, and the validation criteria. However, the literature addresses this phenomenon in a dispersed way, under terms such as communication, coordination, alignment, negotiation, common understanding, and shared mental models, which makes it difficult to use as operational guidance for requirements elicitation, specification, and validation. This undergraduate thesis synthesizes a systematic literature review and a characterization of these activities based on the ISO/IEC/IEEE 29148:2018 standard, and from there adapts the THUNDERS process into *THUNDERS-Master*, a lightweight version aimed at building, monitoring, and validating shared understanding in requirements engineering. The adaptation is carried out through Situational Method Engineering (SME) guided by method chunks, complemented with contextual tailoring criteria and cognitive-load reduction. THUNDERS-Master preserves the three-phase logic of THUNDERS —Beginning, Developing, and Measuring— but translates it into a minimum viable process of eleven operational activities (A1–A11), with simplified roles, minimum artifacts, observable evidence, contextual activation criteria, and explicit traceability to the original process. The result is presented as a formal working specification, ready for expert review, methodological walkthrough, and subsequent piloting; it is not yet presented as an empirically validated process.

Keywords: Requirements engineering, Shared understanding, Situational Method Engineering, Process adaptation, THUNDERS, THUNDERS-Master.

Índice general

Dedicatoria	I
Agradecimientos	II
Resumen	III
Abstract	IV
1. Introducción	1
1.1. Generalidades	1
1.2. Motivación	1
1.3. Planteamiento del problema	2
1.3.1. Descripción del problema	2
1.3.2. Pregunta de investigación	3
1.4. Justificación	3
1.4.1. ¿Por qué se necesita un proceso para el entendimiento compartido en IR?	3
1.4.2. ¿Por qué elicitación, especificación y validación?	4
1.4.3. ¿Por qué usar THUNDERS?	4
1.4.4. ¿Por qué validar en un contexto agrícola?	4
1.5. Objetivos	5
1.5.1. Objetivo general	5
1.5.2. Objetivos específicos	5
1.6. Metodología de la investigación	5
1.7. Aportes del proyecto	6
1.8. Organización del documento	6
2. Marco conceptual y estado del arte	8
2.1. Generalidades	8

2.2.	Marco teórico	8
2.2.1.	Ingeniería de requisitos	8
2.2.2.	Elicitación	8
2.2.3.	Especificación	9
2.2.4.	Validación	9
2.2.5.	Trabajo colaborativo	9
2.2.6.	Entendimiento compartido	9
2.2.7.	Goal-Question-Metric (GQM)	10
2.2.8.	THUNDERS como proceso base	10
2.2.9.	Adaptación de procesos	10
2.3.	Trabajos relacionados	11
2.3.1.	Ingeniería de requisitos y entendimiento compartido	11
2.3.2.	Medición del entendimiento en el ciclo de vida del desarrollo	11
2.3.3.	Construcción del entendimiento en metodologías de desarrollo	11
2.3.4.	Colaboración distribuida y factores culturales	12
2.3.5.	Adaptación de procesos y experiencias contextuales	12
2.4.	Revisión sistemática de la literatura	12
2.4.1.	Protocolo y proceso de revisión	12
2.4.2.	Preguntas de investigación	13
2.4.3.	Estrategia de búsqueda	14
2.4.4.	Fuentes de información	15
2.4.5.	Filtros de búsqueda	15
2.4.6.	Criterios de inclusión y exclusión	16
2.4.7.	Resultados del proceso de selección	17
2.4.8.	Análisis por fuente de información	17
2.4.9.	Distribución temporal de las publicaciones	20
2.4.10.	Resultados por pregunta de investigación	21
2.4.11.	Amenazas a la validez	30
2.4.12.	Síntesis de hallazgos	31
2.4.13.	Brecha de investigación	32
3.	Caracterización de las actividades de ingeniería de requisitos	34
3.1.	Generalidades	34
3.2.	Elicitación de requisitos	34
3.2.1.	Propósito	36
3.2.2.	Alcance y entradas	36

3.2.3.	Tareas, técnicas y productos	36
3.2.4.	Roles involucrados	37
3.2.5.	Artefactos y productos de trabajo	37
3.2.6.	Criterios de calidad y salida	37
3.3.	Especificación de requisitos	38
3.3.1.	Propósito	38
3.3.2.	Alcance y entradas	38
3.3.3.	Tareas que caracterizan la actividad	39
3.3.4.	Reglas de calidad del requisito y del conjunto	39
3.3.5.	Criterios de lenguaje y estilo	40
3.3.6.	Roles involucrados	40
3.3.7.	Artefactos, productos de trabajo y criterios de salida	40
3.4.	Validación de requisitos	40
3.4.1.	Propósito	41
3.4.2.	Alcance y entradas	41
3.4.3.	Tareas, técnicas y criterios de salida	41
3.4.4.	Roles involucrados	41
3.4.5.	Artefactos y productos de trabajo	42
3.5.	Elementos transversales	42
4.	Ingeniería de métodos situacional y adaptación de THUNDERS	44
4.1.	Generalidades	44
4.2.	Metodología de investigación	44
4.3.	Análisis del proceso THUNDERS	45
4.4.	Revisión de métodos de adaptación de procesos	46
4.4.1.	Ingeniería de Métodos Situacionales	46
4.4.2.	Method fragments y method chunks	47
4.4.3.	Software process tailoring	47
4.4.4.	Software process lines	48
4.4.5.	Adaptación ágil basada en contexto	48
4.5.	Comparación de enfoques de adaptación	48
4.6.	Método seleccionado	49
4.7.	Decisiones de adaptación y trazabilidad con THUNDERS	50
4.7.1.	Criterios de decisión de adaptación	50

5. THUNDERS-Master como proceso operativo	55
5.1. Generalidades	55
5.2. Encaje de THUNDERS-Master en marcos ágiles	56
5.3. Requisitos del método	57
5.4. Fase 1: Beginning / Preparar	57
5.5. Fase 2: Developing / Construir	58
5.6. Fase 3: Measuring / Validar y cerrar	58
5.7. Roles y responsabilidades	59
5.8. Artefactos mínimos y evidencia observable	59
5.9. Variabilidad contextual y criterios de activación	59
5.10. Estado de madurez y evolución del proceso	62
6. Evaluación de THUNDERS-Master	63
6.1. Generalidades	63
6.2. Plan de evaluación basado en GQM	63
6.3. Estrategia de evaluación	63
6.4. Diseño del estudio de caso en contexto agrícola	64
6.5. Validación por expertos	65
6.6. Walkthrough metodológico	65
6.7. Resultados del pilotaje en contexto agrícola	65
6.8. Análisis de resultados y lecciones aprendidas	66
7. Conclusiones y trabajo futuro	67
7.1. Generalidades	67
7.2. Conclusiones	67
7.3. Contribuciones	68
7.4. Limitaciones	68
7.5. Trabajo futuro	68
7.6. Productos de investigación	69

Índice de figuras

2.1. Flujo de selección de estudios de la revisión sistemática.	17
2.2. Flujo de selección de estudios en ScienceDirect.	18
2.3. Flujo de selección de estudios en Scopus.	19
2.4. Flujo de selección de estudios en Web of Science.	19
2.5. Frecuencia de publicaciones sobre entendimiento compartido en el dominio de la IR (2015–2026).	20
5.1. Modelo conceptual operativo de THUNDERS-Master.	56

Índice de tablas

2.1. Esquema de extracción de datos y su relación con las preguntas de investigación.	13
2.2. Hallazgos de la RSL y su implicación para THUNDERS-Master.	32
2.3. De la brecha identificada a la justificación de adaptar THUNDERS.	33
3.1. Caracterización operativa de las actividades de ingeniería de requisitos. . .	35
3.2. Tareas características de la elicitación.	37
3.3. Tareas características de la especificación.	39
3.4. Tareas características de la validación.	42
3.5. Elementos transversales a las tres actividades.	43
4.1. Comparación de métodos de adaptación frente al caso THUNDERS-Master.	49
4.2. Guía de aplicación de la SME orientada por chunks para derivar THUNDERS-Master.	51
4.3. Criterios de decisión para adaptar los componentes de THUNDERS.	52
4.4. Mapeo inicial de elementos de THUNDERS hacia actividades de IR en THUNDERS-Master.	53
4.5. Trazabilidad resumida entre THUNDERS y THUNDERS-Master.	54
5.1. Encaje de las actividades de THUNDERS-Master en ceremonias ágiles. . .	57
5.2. Especificación resumida de las actividades de THUNDERS-Master.	60
5.3. Roles simplificados en THUNDERS-Master.	61
5.4. Criterios de activación contextual en THUNDERS-Master.	61
5.5. Ruta de evolución prevista para THUNDERS-Master.	62
6.1. Plan de evaluación propuesto para THUNDERS-Master (GQM).	64

Capítulo 1

Introducción

1.1. Generalidades

La ingeniería de requisitos (IR) cumple una función mediadora entre el dominio del problema y el dominio de la solución. Su propósito es descubrir, negociar, documentar, verificar, validar y gestionar requisitos que expresen necesidades reales de los stakeholders, restricciones del contexto y condiciones de aceptación establecidas por estándares internacionales [1], actuando como una hoja de ruta para todo el ciclo de desarrollo [2]. Esta función no es únicamente técnica: en una sesión de requisitos convergen usuarios finales, clientes, representantes de negocio, analistas, desarrolladores, testers, operadores y expertos de dominio, cada uno con vocabularios, prioridades y niveles de autoridad diferentes. Por ello, la calidad de los requisitos depende tanto de la forma del artefacto como del grado en que los participantes logran compartir el significado que dicho artefacto representa [3].

En este trabajo, el *entendimiento compartido* se entiende como el grado en que los participantes convergen en la interpretación de un objeto de entendimiento, comparten una perspectiva común y coordinan acciones alrededor de un objetivo común [4]. En IR, ese objeto puede ser un problema, un concepto del dominio, una necesidad, un requisito, una restricción, un supuesto, un escenario, una historia de usuario o un criterio de aceptación. Cuando esa convergencia es insuficiente, el equipo puede creer que está de acuerdo aunque cada participante interprete de manera distinta el alcance, el valor, la prioridad o la forma de validar un requisito.

1.2. Motivación

Las fallas de entendimiento compartido no siempre son visibles de inmediato. Pueden quedar ocultas tras una ambigüedad persistente en las especificaciones [5], supuestos

no declarados, conflictos no resueltos, pérdida de contexto, requisitos incompletos o problemas operativos derivados de una trazabilidad débil [6]. En la elicitación, estas fallas aparecen cuando se consulta a los stakeholders incorrectos o cuando las necesidades se capturan como frases aisladas sin fuente, motivación ni contexto. En la especificación, se manifiestan cuando las necesidades se transforman en requisitos ambiguos, no verificables o desconectados de su origen. En la validación, surgen cuando los participantes aprueban un requisito por su forma textual, pero no porque todos lo interpreten de la misma manera.

THUNDERS ofrece una base conceptual relevante para atender este problema, porque fue diseñado para mejorar actividades colaborativas mediante la construcción, el monitoreo y la asistencia del entendimiento compartido [7]. Su estructura considera fases, tareas, roles, productos de trabajo, mecanismos de monitoreo y guías de asistencia. Sin embargo, su aplicación directa a la IR no es trivial: las validaciones previas muestran que el proceso original es completo y metodológicamente robusto, pero en entornos dinámicos resulta extenso, difícil de aplicar y cognitivamente demandante [8]. En contextos de requisitos —donde muchas actividades ocurren en talleres breves, reuniones de refinamiento, entrevistas o revisiones con stakeholders— un proceso pesado dificulta la adopción.

1.3. Planteamiento del problema

1.3.1. Descripción del problema

La ingeniería de requisitos es una función interdisciplinaria que media entre los dominios del adquirente y el proveedor para establecer y mantener los requisitos que debe cumplir un sistema, software o servicio de interés [1]. Esta función comprende actividades sistemáticas de descubrimiento, elicitación, análisis, especificación, verificación, validación, comunicación, documentación y gestión de requisitos, que se ejecutan de forma iterativa a lo largo del ciclo de vida y que demandan la participación coordinada de clientes, usuarios, desarrolladores, operadores y responsables de negocio para construir y mantener un entendimiento compartido de las necesidades y restricciones del sistema. Sin este entendimiento, los grupos suelen tener interpretaciones distintas de lo que deben hacer y del problema a resolver, lo que genera baja colaboración y resultados no deseados [7].

Construir este entendimiento es difícil debido a la diversidad de perspectivas, conocimientos y experiencias de quienes definen los requisitos, a la complejidad de los contextos y conceptos que cada actor maneja, a la carga cognitiva asociada al esfuerzo de interpretar, coordinar y mantener una comprensión común, y a la ausencia de procesos que aseguren su mantenimiento en el tiempo [7]. Esta dificultad no surge de forma aislada: se ve influida

por condiciones organizacionales, culturales y técnicas. Durante la elicitación y el análisis pueden aparecer criterios con definiciones ambiguas o contradictorias que dificultan acuerdos estables [9]; factores culturales como la *distancia de poder* limitan la participación de los stakeholders con menor jerarquía y sesgan la negociación [10]; y, en equipos distribuidos, las distancias geográfica, temporal y sociocultural exigen mecanismos explícitos para sostener una comprensión común del producto, sus requisitos y su entorno [11].

Frente a esta dificultad se retoma el proceso THUNDERS, construido mediante Ingeniería de Métodos Situacionales y especificado en SPEM 2.0 a dos niveles —conceptual y tecnológico— para diseñar, ejecutar y validar actividades de resolución de problemas con grupos heterogéneos [7]. Las experiencias de validación reportadas muestran que THUNDERS es completo y útil y que apoya la construcción de entendimiento compartido; no obstante, también es un proceso largo, difícil de usar y generador de alta carga cognitiva, que aún requiere identificar las tareas obligatorias según el contexto y derivar versiones más simplificadas que conserven sus aportes pero con una complejidad adecuada para entornos reales [7, 8]. El problema, por tanto, no es la ausencia de mecanismos en la literatura, sino la falta de una ruta operativa, ligera y trazable que permita construir, monitorear y validar el entendimiento compartido durante la elicitación, la especificación y la validación de requisitos.

1.3.2. Pregunta de investigación

¿Cómo puede apoyarse la construcción y el mantenimiento del entendimiento compartido durante las actividades de elicitación, especificación y validación de requisitos de un producto software, a partir del proceso THUNDERS?

1.4. Justificación

1.4.1. ¿Por qué se necesita un proceso para el entendimiento compartido en IR?

En la práctica, es frecuente que el entendimiento compartido se pierda o resulte incompleto, porque los actores tienen diferencias de conocimiento, experiencia, lenguaje técnico y contexto [11]. Estas diferencias llevan a interpretaciones distintas de lo que se quiere lograr y, ante la ausencia de un seguimiento sistemático de lo entendido y acordado, el grupo avanza con concepciones parciales o contradictorias que derivan en retrabajo. No basta con documentar los requisitos: es necesario asegurar que todas las partes compren-

dan lo mismo. Sin un proceso explícito que promueva y verifique esta alineación, el riesgo de malentendidos persiste a lo largo de toda la IR [1].

1.4.2. ¿Por qué elicitación, especificación y validación?

Estas tres actividades constituyen el núcleo donde se construye, negocia y consolida el significado de los requisitos. En la elicitación los actores expresan necesidades inicialmente implícitas, incompletas o contradictorias; en la especificación esas necesidades se transforman en representaciones que buscan reducir ambigüedad; y en la validación se confirma que lo documentado refleja la intención de los interesados y que todos interpretan los requisitos de manera coherente. Actividades posteriores como diseño, implementación y pruebas materializan y verifican una solución basada en requisitos ya definidos, pero no son el espacio óptimo para construir entendimiento compartido desde cero: detectar discrepancias allí implica mayores costos de corrección y retrabajo [7, 1].

1.4.3. ¿Por qué usar THUNDERS?

THUNDERS ofrece una base compuesta por roles, tareas, productos de trabajo, momentos de medición y mecanismos de validación, cuyos resultados han mostrado mejoras en la colaboración y en el logro del entendimiento compartido [7]. Esto lo convierte en un punto de partida sólido en lugar de diseñar un proceso completamente nuevo. La justificación no es crear un proceso desde cero, sino adaptar y simplificar los aportes de THUNDERS para ajustarlos a la IR en contextos reales, garantizando que el entendimiento compartido sea visible, comprobable y sostenible.

1.4.4. ¿Por qué validar en un contexto agrícola?

Se propone validar el proceso en un contexto agrícola por las particularidades comunicativas, colaborativas y sociotécnicas del sector agropecuario, donde la adopción de nuevas herramientas depende de la capacidad de los actores para construir significados compartidos y consensuar decisiones sobre los requisitos. Frente a los desafíos del cambio climático y la incorporación de tecnologías digitales, estos entornos exigen interacciones efectivas entre productores, asesores, técnicos, investigadores y diseñadores, con diferencias de lenguaje, marcos conceptuales y percepciones del riesgo que incrementan la probabilidad de malentendidos [12]. Se ha documentado que múltiples proyectos tecnológicos del sector fracasan por no considerar adecuadamente las percepciones, lenguajes y contextos de los usuarios finales [13], y que una comprensión colectiva explícita favorece la toma de de-

ciones, la confianza y la adopción de tecnologías resilientes [14]. Por ello, este dominio fortalece la utilidad, transferibilidad y adaptabilidad del proceso propuesto.

1.5. Objetivos

1.5.1. Objetivo general

Adaptar el proceso THUNDERS para apoyar la construcción del entendimiento compartido en las actividades de elicitación, especificación y validación de requisitos de la ingeniería de requisitos, integrándolo como apoyo al proceso de desarrollo de un producto de software.

1.5.2. Objetivos específicos

1. Caracterizar, de acuerdo con la literatura, los elementos de las actividades de elicitación, especificación y validación de requisitos que requieren construir entendimiento compartido.
2. Identificar del proceso THUNDERS los componentes, principios y mecanismos susceptibles de adaptación para apoyar las actividades de elicitación, especificación y validación de la IR.
3. Definir THUNDERS-Master como un proceso adaptado que permita construir el entendimiento compartido durante las actividades de elicitación, especificación y validación de requisitos de software.
4. Evaluar, a través de un estudio de caso en un contexto agrícola, la utilidad del proceso adaptado para la construcción del entendimiento compartido en un equipo de desarrollo de un producto de software orientado a dicho contexto.

1.6. Metodología de la investigación

La investigación adopta una metodología de carácter aplicado, iterativo y orientado a la evidencia [15], adecuada para adaptar y validar procesos en contextos reales. La iteración se materializa en la posibilidad de refinar la propuesta con base en la evidencia recogida durante el pilotaje. Para definir y justificar las métricas de evaluación se adopta el enfoque Goal-Question-Metric (GQM) [16], que permite operacionalizar el constructo de entendimiento compartido en indicadores observables y medibles, garantizando trazabilidad entre objetivos, preguntas de evaluación y métricas. El trabajo se organiza en cuatro

fases.

Fase 1 — Exploración. Revisión sistemática de la literatura (RSL) para caracterizar los elementos de elicitación, especificación y validación que demandan entendimiento compartido (Objetivo 1), y análisis documental de THUNDERS para identificar los componentes adaptables a IR (Objetivo 2). Su producto es un informe que consolida la caracterización y el mapeo de elementos de THUNDERS.

Fase 2 — Formulación del proceso. Búsqueda dirigida de modelos de adaptación, selección del más pertinente y su aplicación estructurada (criterios de adaptación, mapeo de elementos, reglas de transformación y especificación de la versión adaptada), junto con el diseño del plan de medición GQM (Objetivo 3). La fase cierra con un taller de validación con expertos y usuarios clave para refinar la versión a pilotar.

Fase 3 — Evaluación. Implementación piloto de THUNDERS-Master en un estudio de caso en contexto agrícola (Objetivo 4), mediante sesiones de elicitación, especificación y validación, ejecutando el plan GQM. Los datos se analizan con estadística descriptiva (percepción) y análisis temático (información cualitativa), produciendo un registro de lecciones aprendidas y ajustes al proceso.

Fase 4 — Documentación y transferencia. Consolidación de los resultados en la monografía y sustentación del trabajo de grado.

1.7. Aportes del proyecto

El aporte se centra en definir un proceso adaptado para construir entendimiento compartido específicamente en elicitación, especificación y validación de requisitos. THUNDERS-Master no sustituye prácticas existentes: las complementa con principios de ingeniería de la colaboración y análisis contextual. En el ámbito académico, amplía la comprensión teórica y práctica del entendimiento compartido en IR; en el industrial, aporta un proceso formal pero liviano que reduce ambigüedades, retrabajo y errores de interpretación; y en el investigativo, extiende la aplicación de THUNDERS al dominio de la IR, generando evidencia empírica sobre cómo el entendimiento compartido puede construirse, monitorearse y evaluarse. La validación en un contexto agrícola aporta una perspectiva aplicada en un escenario con alta diversidad técnica, disciplinar y comunicativa [14, 13].

1.8. Organización del documento

El resto del documento se organiza así. El Capítulo 2 presenta el marco teórico, los trabajos relacionados y la revisión sistemática de la literatura. El Capítulo 3 caracteriza las

actividades de elicitación, especificación y validación. El Capítulo 4 describe la Ingeniería de Métodos Situacionales y las decisiones de adaptación de THUNDERS. El Capítulo 5 especifica THUNDERS-Master como proceso operativo (fases, actividades, roles, artefactos y variabilidad contextual). El Capítulo 6 presenta el plan y los resultados de la evaluación. Finalmente, el Capítulo 7 expone conclusiones, contribuciones, limitaciones y trabajo futuro.

Capítulo 2

Marco conceptual y estado del arte

2.1. Generalidades

Este capítulo establece las bases conceptuales del trabajo y sintetiza la evidencia disponible. Primero define los conceptos de ingeniería de requisitos y entendimiento compartido; luego describe THUNDERS como proceso base; y finalmente resume la revisión sistemática de la literatura (RSL) que sirvió como insumo de diseño para THUNDERS-Master.

2.2. Marco teórico

2.2.1. Ingeniería de requisitos

La ingeniería de requisitos (IR) es un proceso sistemático y repetible para identificar, analizar, documentar y validar lo que un sistema debe hacer y bajo qué condiciones, de modo que todos los interesados compartan una base común para el desarrollo y la aceptación del sistema. La norma ISO/IEC/IEEE 29148:2018 define la IR como el conjunto de actividades que transforman las necesidades de los interesados en requisitos verificables y gestionables a lo largo del ciclo de vida —elicitar, analizar/negociar, especificar, validar y gestionar— [1]. Este trabajo se delimita a elicitación, especificación y validación porque son los momentos donde el significado de los requisitos se construye, se formaliza y se confirma.

2.2.2. Elicitación

Es la actividad en la que se identifican y recopilan las necesidades, expectativas y restricciones de los stakeholders sobre el sistema. Según ISO/IEC/IEEE 29148, busca

capturar requisitos desde distintas fuentes y perspectivas, por lo que requiere preparación e identificación de fuentes para obtener información que sirva de base para su análisis y documentación [1]. No es una simple recolección de información, sino una construcción inicial de significado [17].

2.2.3. Especificación

Corresponde a la actividad en la que los requisitos elicitados se documentan y estructuran de forma clara, completa y verificable, para servir como base de comunicación y acuerdo entre stakeholders y equipo de desarrollo. Los requisitos se formulan con redacción precisa y medible, manteniendo consistencia y trazabilidad para facilitar su revisión y uso en etapas posteriores [1, 18].

2.2.4. Validación

Es la actividad mediante la cual se confirma, junto con los stakeholders, que los requisitos documentados representan correctamente sus necesidades y son adecuados para el propósito del sistema. Se realiza a través de revisiones y retroalimentación con los interesados, con el fin de detectar inconsistencias, ambigüedades u omisiones antes de avanzar [1].

2.2.5. Trabajo colaborativo

El trabajo colaborativo se entiende como la contribución conjunta de ideas, experiencias y conocimientos de un grupo orientado a una meta común. Su propósito no es solo optimizar resultados, sino generar conocimiento colectivo y fortalecer el entendimiento compartido del problema [7]. Se caracteriza por la interdependencia positiva: el éxito depende de integrar los aportes en una construcción conjunta, más allá del reparto de tareas, y requiere comunicación efectiva, escucha activa y toma de decisiones compartida [19].

2.2.6. Entendimiento compartido

El entendimiento compartido (*Shared Understanding*, SU) se construye a partir del intercambio de información y la interacción social, y se refleja en acuerdos sobre procesos, significados y orden de las actividades, permitiendo coordinar conductas hacia objetivos comunes [4, 7]. En IR se materializa cuando clientes, usuarios, analistas, diseñadores y demás interesados interpretan de manera equivalente las necesidades, restricciones y criterios de calidad, lo que reduce ambigüedades y retrabajo [9]. La literatura lo aborda de forma dispersa bajo términos como comunicación, coordinación, alineación, negociación,

entendimiento común y modelos mentales compartidos, lo que limita su uso como guía operativa.

2.2.7. Goal-Question-Metric (GQM)

GQM es un método dirigido por objetivos para definir e interpretar mediciones en ingeniería de software: primero se establece un objetivo de medición, luego se descompone en preguntas que permiten evaluar su cumplimiento y, finalmente, se definen métricas concretas para responder esas preguntas. Así, asegura que las métricas recolectadas estén alineadas con las necesidades de información del proyecto [16].

2.2.8. THUNDERS como proceso base

THUNDERS es un proceso de trabajo colaborativo orientado a construir, medir, monitorear y asistir el entendimiento compartido durante actividades de resolución de problemas [7]. Fue construido con Ingeniería de Métodos Situacionales y formalizado en SPEM 2.0, a nivel conceptual y tecnológico, y organiza fases, actividades, roles, tareas, productos de trabajo y mecanismos de validación. Su estructura se organiza en tres fases: *Beginning* (diseñar y especificar la actividad colaborativa), *Developing* (ejecutar la actividad y construir el entendimiento) y *Measuring* (validar la solución, evaluar desempeño y cerrar), con mecanismos transversales de monitoreo y asistencia. No obstante, su validación reporta que, aunque es completo y útil, también es extenso, difícil de aplicar y generador de alta carga cognitiva [8].

2.2.9. Adaptación de procesos

La adaptación de procesos es el ajuste sistemático de un proceso para adecuarlo a las condiciones específicas de un proyecto u organización, manteniendo su eficacia ante variables como recursos, cultura, capacidades del equipo, herramientas o requisitos de calidad [20]. Adaptar no es solo modificar tareas o roles, sino asegurar consistencia con los objetivos del proceso. Entre los pasos comúnmente considerados se incluyen: diagnóstico y contextualización, definición de criterios de adaptación, mapeo del proceso original, diseño de reglas de transformación, documentación del proceso adaptado y validación con mejora iterativa [20].

2.3. Trabajos relacionados

La revisión de la literatura identificó investigaciones que, desde distintos enfoques, buscan construir, mantener y medir el entendimiento compartido en la IR.

2.3.1. Ingeniería de requisitos y entendimiento compartido

Los trabajos de esta línea muestran que contar con artefactos compartidos y representaciones comunes mejora la comunicación y reduce la ambigüedad en las fases iniciales. El uso de *personas* y escenarios narrativos facilita entender las motivaciones de los usuarios [21], y acordar y mantener visibles los criterios de calidad disminuye errores de interpretación y deuda técnica [18]. Sin embargo, muchos enfoques abordan el SU de manera fragmentada —centrados en artefactos, modelos o dinámicas sociales por separado— sin integrarlos en un proceso coherente y continuo.

2.3.2. Medición del entendimiento en el ciclo de vida del desarrollo

Una segunda línea estudia cómo evidenciar o medir el SU según la fase del ciclo de vida. En fases tempranas se emplean grafos de conocimiento y modelos semánticos para evaluar si los actores comparten significados equivalentes sobre el dominio; al avanzar, la medición se apoya en la trazabilidad y la consistencia entre artefactos. Pese a ello, la literatura coincide en que aún no existe una metodología estandarizada para un seguimiento continuo del SU dentro de la IR [7, 1], lo que justifica incorporar mecanismos formales de monitoreo y verificación.

2.3.3. Construcción del entendimiento en metodologías de desarrollo

Los paradigmas de desarrollo influyen en cómo los equipos construyen y mantienen el SU. Los enfoques empíricos y ágiles ofrecen más oportunidades para co-construir modelos mentales compartidos [22], pero los estudios sobre adopciones ágiles a gran escala muestran que, sin estructuras claras de coordinación, trazabilidad y roles de apoyo, el entendimiento tiende a fragmentarse entre equipos y niveles organizacionales [23]. El reto no es oponer «ágil» frente a «tradicional», sino disponer de procesos que hagan explícita la construcción, el monitoreo y la verificación del SU.

2.3.4. Colaboración distribuida y factores culturales

Los estudios sobre colaboración distribuida examinan cómo la dispersión geográfica, los husos horarios y las diferencias culturales afectan la coordinación. Cuando los miembros trabajan desde distintos lugares surgen riesgos de pérdida de contexto, retrasos en la retroalimentación y desalineación del SU [11]. Prácticas de integración continua, repositorios colaborativos y reuniones de sincronización ayudan a mitigarlos. En cuanto a lo cultural, las diferencias jerárquicas y de poder influyen en la participación y la calidad de la negociación: las estructuras rígidas limitan la retroalimentación, mientras que las culturas más horizontales fomentan la confianza y la validación colectiva [10].

2.3.5. Adaptación de procesos y experiencias contextuales

La adaptación de procesos es una estrategia clave para ajustar metodologías a contextos ágiles o de pequeña escala, evaluando los cambios no solo por su impacto técnico sino por su influencia en la coordinación y el entendimiento del equipo [23, 20]. De forma complementaria, los resultados de THUNDERS en escenarios colaborativos son positivos, pero evidencian la necesidad de versiones más ligeras y adaptadas a equipos reales, con artefactos simplificados y mecanismos de validación claros [7]. Esta brecha es justamente la que THUNDERS-Master busca atender.

2.4. Revisión sistemática de la literatura

La RSL se condujo siguiendo el proceso de mapeo sistemático propuesto por Petersen et al. [15]. Su objetivo fue identificar, organizar y sintetizar estudios primarios que reportan rupturas, situaciones críticas, mecanismos de apoyo y formas de evaluación del entendimiento compartido en la elicitación, especificación y validación de requisitos.

2.4.1. Protocolo y proceso de revisión

La revisión siguió un protocolo definido a priori, organizado en las fases del mapeo sistemático [15]: *(i)* planificación, que fijó el objetivo, las preguntas de investigación y los criterios de inclusión y exclusión; *(ii)* búsqueda, con una cadena estructurada aplicada a las bases seleccionadas; *(iii)* cribado en dos etapas, primero por título y resumen y luego por texto completo; *(iv)* extracción de datos mediante un esquema alineado con las preguntas de investigación; y *(v)* síntesis temática de los estudios primarios. La búsqueda, los criterios

y el flujo de selección se detallan en las subsecciones siguientes; aquí se describe el cribado y la extracción que sostienen la trazabilidad entre la evidencia y los hallazgos.

El cribado se aplicó de forma secuencial. En la primera etapa se evaluaron título, resumen y palabras clave frente a los criterios de inclusión y exclusión, y se descartaron los estudios claramente fuera del dominio de requisitos o sin relación con el entendimiento compartido. En la segunda etapa se leyó el texto completo de los estudios restantes y se conservaron solo aquellos que aportaban evidencia útil para al menos una pregunta de investigación. Ambas etapas fueron realizadas de forma independiente por los dos autores del trabajo; los desacuerdos sobre la inclusión de un estudio se resolvieron por discusión directa entre ambos y, cuando el caso lo ameritaba, mediante consulta a la directora del trabajo de grado. Adicionalmente, la directora revisó el proceso de cribado y sus resultados como control de calidad de la selección. La extracción se apoyó en un esquema que asocia cada campo con la pregunta de investigación que alimenta (Tabla 2.1), de modo que la síntesis por pregunta de investigación se construyó sobre datos recolectados de forma homogénea y no sobre lecturas *ad hoc* de cada estudio.

Tabla 2.1: Esquema de extracción de datos y su relación con las preguntas de investigación.

Campo extraído	Descripción	Pregunta
Metadatos del estudio	Autores, año, fuente, tipo de estudio y actividad(es) de IR abordada(s).	Todas
Problema o ruptura de entendimiento	Fallas de comunicación, ambigüedad o desalineación reportadas.	RQ1
Tarea, artefacto o situación	Dónde se identifica la necesidad de construir entendimiento compartido.	RQ2
Momento crítico	Punto del proceso donde el significado es inestable y debe negociarse.	RQ3
Mecanismo, técnica o enfoque	Solución propuesta o usada para construir o mantener el entendimiento.	RQ4
Métrica, indicador o instrumento	Forma de medir o evaluar el entendimiento y su momento de aplicación.	RQ5

2.4.2. Preguntas de investigación

- **RQ1.** ¿Qué problemas de comunicación o desalineaciones del entendimiento compartido se reportan durante la elicitación, especificación y validación?
- **RQ2.** ¿En qué tareas, artefactos y situaciones se identifica la necesidad de construir entendimiento compartido entre stakeholders?
- **RQ3.** ¿Cuáles son los momentos más críticos para lograr entendimiento compartido en cada actividad y por qué?

- **RQ4.** ¿Qué mecanismos, técnicas o enfoques se han propuesto o usado para apoyar la construcción y el mantenimiento del entendimiento compartido?
- **RQ5.** ¿Qué métricas, indicadores e instrumentos se han propuesto para medir o evaluar el entendimiento compartido, y en qué momentos (antes, durante o después)?

En particular, en la RQ5, *medir* se entiende como la identificación o recolección de métricas e indicadores observables asociados al entendimiento compartido, mientras que *evaluar* implica interpretar esos indicadores o valorar la evidencia mediante instrumentos, criterios, escalas, cuestionarios o juicio de expertos. Asimismo, los momentos de aplicación antes, durante y después se entienden con respecto a cada actividad analizada, es decir, antes de iniciar, durante la ejecución o después de completar la elicitación, la especificación o la validación de requisitos. Estas preguntas son consistentes con el objetivo de la revisión, pues permiten caracterizar el entendimiento compartido desde cinco dimensiones complementarias: las rupturas o problemas reportados, las tareas y artefactos donde se requiere, los momentos críticos en que debe lograrse, los mecanismos propuestos para apoyarlo y los enfoques usados para medirlo o evaluarlo.

2.4.3. Estrategia de búsqueda

La búsqueda se estructuró alrededor de tres conceptos centrales: *Requirements Engineering*, *Shared Understanding* y *Software Development*. La cadena general de búsqueda fue:

```
("Requirements Engineering" OR "Elicitation" OR  
  "Requirements Analysis")  
AND ("Shared Understanding" OR "Common Understanding" OR  
  "Cognitive Alignment")  
AND ("Software Development")
```

La cadena no incluyó términos específicos por subactividad (elicitación, especificación, validación) con el fin de maximizar la sensibilidad de la recuperación: dado que el entendimiento compartido aparece frecuentemente fragmentado en la literatura bajo términos relacionados como comunicación, alineación o coordinación, una cadena más restrictiva habría implicado una pérdida importante de estudios primarios potencialmente relevantes. Por ello, la especificidad temática se garantizó mediante la aplicación rigurosa de los criterios de inclusión (en especial IC1 e IC2) durante el cribado y la evaluación de elegibilidad, de modo que solo se seleccionaran finalmente estudios que aportaran evidencia concreta sobre esas actividades.

2.4.4. Fuentes de información

La búsqueda se condujo en tres bases de datos con amplia cobertura en Ingeniería de Software y áreas afines: *ScienceDirect*, *Scopus* y *Web of Science*. La búsqueda inicial, antes de aplicar filtros, eliminar duplicados y evaluar los criterios de inclusión y exclusión, identificó los siguientes registros: *ScienceDirect*, 826 registros iniciales; *Scopus*, 420 registros iniciales; y *Web of Science*, 116 registros iniciales. Estos valores corresponden al número de registros recuperados en la primera ejecución de la cadena de búsqueda en cada base. La selección de estas fuentes responde a su reconocida relevancia para recuperar publicaciones arbitradas en los campos de la computación, la ingeniería y el desarrollo de software.

2.4.5. Filtros de búsqueda

Para delimitar el corpus de estudios y mantener la revisión alineada con el objetivo planteado, se aplicaron los siguientes filtros de búsqueda:

- **Periodo temporal:** 2015–2026.
- **Tipo de documento:** artículos de investigación.
- **Idioma:** inglés.
- **Acceso:** abierto (*open access*).
- **Áreas temáticas:** *Computer Science*, *Engineering*, *Software Engineering*, *Requirements Engineering* y términos relacionados con *Shared Understanding*.

El periodo 2015–2026 se definió para recuperar evidencia de la última década, considerando que el entendimiento compartido en IR ha ganado relevancia particular en entornos de desarrollo de software contemporáneos, caracterizados por mayor colaboración entre stakeholders, distribución geográfica, mediación tecnológica y evolución continua de los requisitos [24]. En este tipo de contextos, las dificultades de comunicación y coordinación pueden afectar la construcción de una interpretación común entre participantes [25]. Asimismo, este rango permite incluir estudios recientes asociados a enfoques ágiles, desarrollo a gran escala y prácticas de especificación y validación que dependen de la alineación entre requisitos, artefactos, pruebas y decisiones de implementación [26]. Por tanto, el periodo seleccionado ofrece un equilibrio entre actualidad, amplitud suficiente del corpus y pertinencia con el objetivo de analizar la construcción, el mantenimiento y la evaluación del entendimiento compartido en las actividades de IR.

2.4.6. Criterios de inclusión y exclusión

Criterios de inclusión

Un estudio se incluyó si cumplía todos, o al menos la mayoría, de los siguientes criterios:

- **IC1.** El estudio aborda explícitamente actividades de Ingeniería de Requisitos (IR), en particular elicitación, especificación, análisis o validación de requisitos.
- **IC2.** El estudio reporta problemas, situaciones, artefactos, mecanismos, prácticas, métricas o instrumentos relacionados con el entendimiento compartido, la alineación conceptual, el entendimiento común o fenómenos equivalentes entre *stakeholders*.
- **IC3.** El estudio se sitúa en el contexto del desarrollo de software, la ingeniería de software o los sistemas de software.
- **IC4.** El estudio presenta evidencia suficiente para responder al menos una de las preguntas de investigación definidas.
- **IC5.** El estudio es una publicación primaria, revisada por pares, escrita en inglés y disponible en texto completo.
- **IC6.** El estudio fue publicado dentro del periodo 2015–2026.

Criterios de exclusión

Un estudio se excluyó si cumplía al menos uno de los siguientes criterios:

- **EC1.** El estudio no se centra en IR, o aborda el entendimiento compartido en actividades distintas de la elicitación, especificación o validación sin establecer una relación explícita con alguna de ellas.
- **EC2.** El estudio aborda la colaboración, la comunicación o el trabajo en equipo de forma general, sin vincularlos con los requisitos de software o con actividades concretas de IR.
- **EC3.** El estudio pertenece a dominios distintos del desarrollo de software y no ofrece evidencia transferible al contexto de la IR.
- **EC4.** El estudio es una revisión sistemática, un mapeo sistemático, un editorial, un capítulo de libro, un resumen, un póster, una tesis u otra publicación no primaria.
- **EC5.** El estudio no está disponible en acceso abierto, no está escrito en inglés o no está disponible en texto completo.
- **EC6.** El estudio no aporta información útil para responder ninguna de las preguntas de investigación, aun cuando mencione términos relacionados con comunicación, colaboración o entendimiento.
- **EC7.** El estudio es un duplicado recuperado en más de una base de datos.

2.4.7. Resultados del proceso de selección

La aplicación de la estrategia recuperó inicialmente 1.362 registros. Tras aplicar los filtros se redujeron a 153 estudios y, eliminados los duplicados, quedaron 144 estudios únicos (101 de ScienceDirect, 24 de Scopus y 19 de Web of Science). La revisión de título y resumen excluyó 95 estudios y seleccionó 49 para lectura de texto completo. Finalmente se seleccionaron **29 estudios primarios** (14 de ScienceDirect, 7 de Scopus y 8 de Web of Science). La Figura 2.1 resume el flujo de selección.

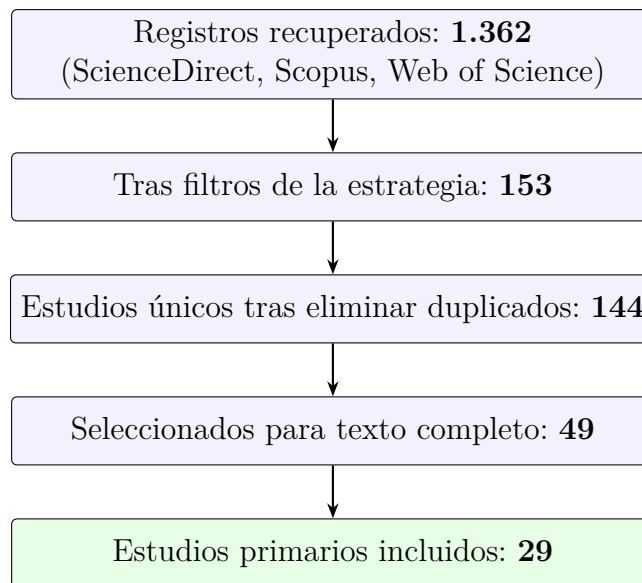


Figura 2.1: Flujo de selección de estudios de la revisión sistemática.

El alto número de exclusiones no constituye una debilidad de la revisión, sino un efecto esperado de una estrategia de búsqueda deliberadamente amplia, orientada a minimizar el riesgo de omitir estudios relevantes en un dominio aún disperso y conceptualmente heterogéneo. Asimismo, la reducción relativamente baja debida a duplicados (de 153 a 144 registros) indica que, aunque existió cierto solapamiento entre las bases consultadas, cada una aportó registros parcialmente diferenciados.

2.4.8. Análisis por fuente de información

Para observar con mayor precisión el comportamiento de cada fuente durante el proceso de selección, los resultados no se agrupan en una única figura general, sino que se presentan mediante tres diagramas independientes, uno por base de datos consultada.

La Figura 2.2 presenta el flujo de selección correspondiente a ScienceDirect. Esta base recuperó el mayor número de registros iniciales (826), lo que evidencia una amplia cober-

tura del dominio de búsqueda; sin embargo, la aplicación sucesiva de los filtros temporal, de tipo de documento, de área temática y de acceso abierto redujo considerablemente el número de estudios potencialmente relevantes, hasta llegar a 14 estudios primarios.

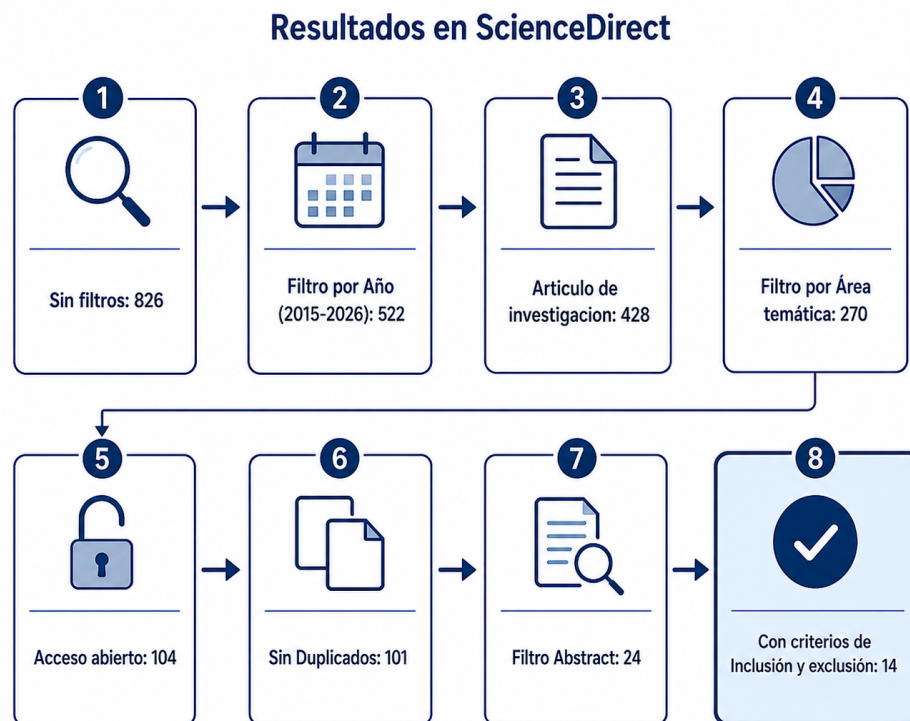


Figura 2.2: Flujo de selección de estudios en ScienceDirect.

La Figura 2.3 muestra el proceso de filtrado de los registros obtenidos en Scopus. En esta base se identificaron inicialmente 420 artículos, sometidos a los mismos criterios de filtrado aplicados en las demás fuentes, de los cuales 7 se consolidaron como estudios primarios.

Finalmente, la Figura 2.4 presenta el flujo correspondiente a Web of Science. Aunque esta base recuperó un número menor de registros iniciales (116), su aporte al conjunto final de estudios fue proporcionalmente relevante, con 8 estudios primarios seleccionados. Esto muestra que un menor número inicial de resultados no implica necesariamente una menor pertinencia de los estudios recuperados.

En cuanto al aporte final de cada base, ScienceDirect contribuyó con 14 estudios primarios, Scopus con 7 y Web of Science con 8. Aunque ScienceDirect recuperó el mayor número de registros iniciales, esta amplitud no se tradujo necesariamente en una mayor proporción de estudios relevantes al final del proceso; Web of Science presentó una recuperación relativamente más precisa para el dominio específico de estudio, mientras que



Figura 2.3: Flujo de selección de estudios en Scopus.



Figura 2.4: Flujo de selección de estudios en Web of Science.

Scopus mostró un comportamiento intermedio.

2.4.9. Distribución temporal de las publicaciones

La Figura 2.5 muestra la frecuencia de publicación de los estudios primarios sobre entendimiento compartido en elicitación, especificación y validación de requisitos durante el periodo 2015–2026. Los resultados muestran una producción científica baja e intermitente entre 2015 y 2020, con valores entre cero y tres publicaciones por año. A partir de 2021 se observa una mayor concentración de estudios, con picos en 2021 y 2023, cada uno con cinco publicaciones. No obstante, esta distribución no permite afirmar la existencia de una tendencia lineal sostenida, sino que sugiere un aumento reciente del interés por el tema.

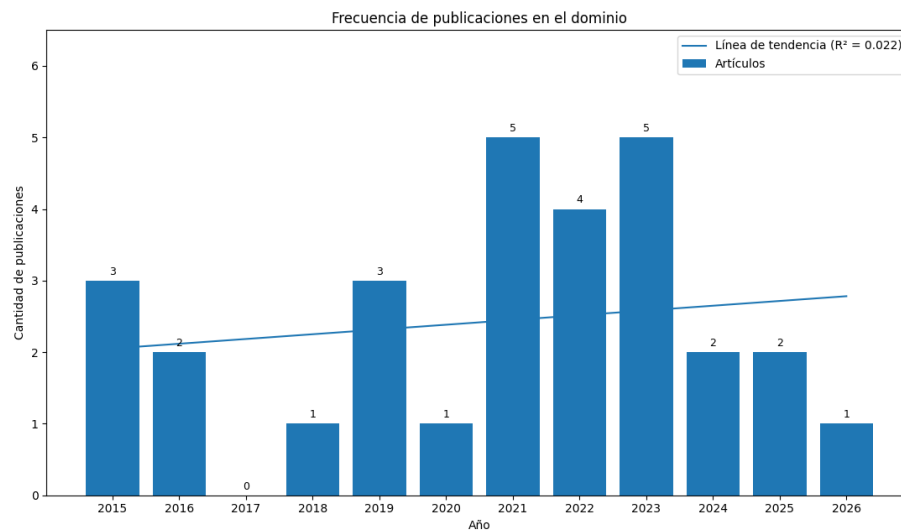


Figura 2.5: Frecuencia de publicaciones sobre entendimiento compartido en el dominio de la IR (2015–2026).

Este comportamiento puede interpretarse como un indicio de la consolidación gradual del estudio del entendimiento compartido en IR, especialmente en los años recientes. La distribución temporal también muestra que el tema no se ha abordado de forma continua ni homogénea a lo largo del periodo analizado, sino que aparece disperso entre los estudios seleccionados. En consecuencia, la frecuencia de publicación identificada respalda la idea de que el entendimiento compartido en IR es un tema vigente y en desarrollo conceptual, aunque todavía con una producción relativamente limitada y heterogénea, lo que refuerza la necesidad de revisiones sistemáticas enfocadas que permitan organizar y sintetizar la evidencia disponible.

2.4.10. Resultados por pregunta de investigación

RQ1 — Problemas de comunicación y desalineación

La literatura revisada muestra que los problemas de comunicación y las desalineaciones del entendimiento compartido en IR aparecen desde las primeras interacciones entre stakeholders y persisten a lo largo de la elicitación, la especificación y la validación. Una causa inicial recurrente es la falta de un entendimiento común del proceso de negocio y del propósito del sistema, ya que los participantes pueden interpretar de forma distinta qué problema debe resolver el software, cuáles son sus límites y qué resultados se esperan de él [27]. También se reporta una brecha entre la perspectiva de negocio y la perspectiva técnica: mientras algunos stakeholders expresan necesidades en términos de objetivos, procesos o valor para la organización, los perfiles técnicos deben traducir esas necesidades en decisiones de diseño, implementación, pruebas y mantenimiento; cuando esa traducción no se hace explícita, desarrolladores, evaluadores e ingenieros pueden construir interpretaciones distintas del alcance real de los requisitos [24]. A esto se suman diferencias en el lenguaje usado por los participantes: un mismo concepto de dominio puede entenderse de forma distinta entre usuarios, analistas, desarrolladores o evaluadores, especialmente cuando no existe un vocabulario común o se usan términos ambiguos, sinónimos o expresiones específicas de un área [28]. En algunos casos, la propia estructura organizacional contribuye a la desalineación, porque la separación rígida de roles limita la construcción de una visión compartida del producto [3].

En la elicitación, varios estudios muestran que el entendimiento compartido se debilita cuando los stakeholders relevantes no se identifican correctamente o cuando los requisitos se obtienen de actores que no representan adecuadamente las necesidades reales del sistema [29]. También se reportan dificultades cuando los stakeholders no logran expresar claramente lo que necesitan, lo que puede derivar en requisitos vagos, contradictorios o incompletos desde etapas tempranas [30]. En contextos distribuidos, estas dificultades aumentan por barreras de idioma, diferencias culturales y dependencia de mecanismos de traducción que no siempre preservan el significado original con precisión [25]; de forma similar, en entornos globales la distancia geográfica y temporal dificulta la comunicación entre cliente y proveedor [31], mientras que los problemas de coordinación entre equipos distribuidos profundizan la posibilidad de malentendidos [32]. En culturas con alta distancia de poder, algunos stakeholders evitan cuestionar decisiones o corregir interpretaciones erróneas de actores con mayor jerarquía, lo que limita la expresión explícita de las necesidades reales [10].

En la especificación, los problemas más recurrentes se relacionan con la ambigüedad,

la incompletitud y la baja calidad de los artefactos de requisitos. Las especificaciones en lenguaje natural admiten múltiples interpretaciones, de modo que el autor de un requisito y sus lectores —analistas, desarrolladores, evaluadores o stakeholders de negocio— pueden creer que comparten el mismo significado cuando en realidad entienden cosas distintas [33]. En el caso de los requisitos no funcionales, la desalineación puede ser mayor cuando no se discuten explícitamente o se documentan de forma deficiente [18, 34]. Otros estudios indican que la especificación también se ve afectada cuando no existe un lenguaje de dominio común [35], o cuando, al pasar del lenguaje natural a modelos, surgen malentendidos por explicaciones ambiguas del proceso o dificultades para comprender las notaciones de modelado [36].

En la validación, los problemas de desalineación aparecen cuando lo documentado no puede contrastarse claramente con lo que los stakeholders quisieron expresar o acordar. Algunos estudios muestran que esta dificultad se debe a la ausencia de criterios de aceptación claros y compartidos, lo que impide determinar objetivamente si un requisito fue satisfecho [37]. En otros casos, los requisitos se formulan con cualidades vagas o no medibles, lo que dificulta su verificación y deja espacio para el desacuerdo durante la aceptación [30]. También se reporta pérdida de información al traducir requisitos en lenguaje natural a pruebas implementadas en código, especialmente cuando quienes definen las pruebas no participaron en la discusión original del requisito [24]. En entornos ágiles a gran escala, la validación se deteriora además cuando la retroalimentación llega tarde o el equipo pierde contexto [26], y ciertos problemas de calidad del lenguaje natural, como la ambigüedad o la no verificabilidad, dificultan acordar exactamente qué significa el requisito y cómo comprobar su cumplimiento [38].

En conjunto, la literatura sugiere que estos problemas no son solo documentales, sino también sociales, organizacionales y contextuales. La separación de roles y responsabilidades puede dificultar la construcción de una visión compartida del producto [3]; en entornos de desarrollo global, la distribución del conocimiento en múltiples canales y la rotación de participantes dificultan sostener una visión común y estable de los requisitos [39]; y en contextos ágiles a gran escala, la falta de prácticas sólidas de IR puede afectar la construcción y preservación del conocimiento compartido sobre objetivos, requisitos y decisiones del producto [26]. Por ello, los problemas de comunicación en IR no deben entenderse solo como fallas de redacción o transmisión de información, sino como rupturas en la construcción, negociación y mantenimiento del significado compartido entre stakeholders [3].

RQ2 — Tareas, artefactos y situaciones

La literatura revisada muestra que la necesidad de construir entendimiento compartido entre stakeholders aparece de forma transversal en la elicitación, la especificación y la validación de requisitos. Esta necesidad no se refiere únicamente a que cada participante comprenda individualmente la información disponible, sino a que los distintos actores —usuarios, clientes, analistas, desarrolladores, evaluadores y gerentes de negocio— logren una interpretación suficientemente común del problema, los requisitos y los criterios que permitirán validar su cumplimiento [27]. No obstante, la evidencia revisada identifica principalmente tareas, artefactos y situaciones donde el entendimiento compartido es necesario o puede apoyarse, pero no ofrece un procedimiento integrado, secuencial y suficientemente operacionalizado que indique cómo lograrlo sistemáticamente en cada actividad.

En la elicitación, esta necesidad se identifica especialmente en tareas como la identificación de stakeholders relevantes, la recolección de necesidades, la priorización de requisitos y la clarificación de expectativas de usuario [29]. En la identificación de stakeholders, el entendimiento compartido es necesario porque permite reconocer qué actores poseen conocimiento relevante del dominio y evita que la visión inicial del sistema se limite a un grupo reducido de participantes. En la recolección de necesidades, es necesario porque muchas expectativas se expresan de forma informal, incompleta o mediante lenguajes específicos de un área, por lo que deben clarificarse y reinterpretarse colectivamente antes de convertirse en requisitos [30]. La evidencia sitúa esta necesidad en entrevistas, *workshops*, *focus groups* y reuniones de requisitos, donde los participantes deben reconciliar perspectivas de negocio, técnicas y organizacionales para construir una visión inicial compartida del sistema [25]; en esta fase, los artefactos asociados incluyen listas de stakeholders, catálogos de requisitos, historias de usuario iniciales, glosarios de dominio, pizarras compartidas, prototipos en papel y escenarios [40]. Sin embargo, la sola presencia de estos artefactos no permite concluir que el entendimiento compartido ya se haya logrado: una lista de stakeholders puede ayudar a reconocer actores relevantes y un prototipo puede facilitar la discusión de necesidades, pero requieren interacción, discusión y revisión conjunta para contribuir realmente a su construcción.

En la especificación, la necesidad de entendimiento compartido aparece cuando la información recogida durante la elicitación debe convertirse en artefactos estructurados, comprensibles y consistentes para todos los involucrados. La literatura destaca aquí el papel de las historias de usuario, los *feature files* de *Behavior-Driven Development* (BDD), los criterios de aceptación, las plantillas de requisitos y los modelos conceptuales, en tanto funcionan como medios para conectar distintas perspectivas sobre el sistema [24]. Este proceso se hace más visible cuando los requisitos deben reformularse, clasificarse o tradu-

cirse a representaciones más precisas, por ejemplo mediante escenarios *Given/When/Then*, diagramas o *Action Sheets* [41]. No obstante, tampoco en la especificación se identifica un procedimiento completo que garantice que el entendimiento compartido se logra solo mediante la producción de artefactos: una historia de usuario, un modelo o un criterio de aceptación pueden ayudar a estabilizar y comunicar el significado de un requisito, pero también pueden interpretarse de forma distinta si sus términos, condiciones y criterios de cumplimiento no se hacen explícitos.

En la validación, esta necesidad se identifica cuando los stakeholders deben confirmar que lo documentado refleja correctamente la intención original y que existen criterios comunes para evaluar su cumplimiento [37]. Esto se observa en revisiones conjuntas, validación de documentos, aprobación formal de requisitos, *sprint reviews*, validación de modelos y verificación de consistencia entre requisitos y pruebas [42]. Los artefactos más ligados a esta fase incluyen criterios de aceptación, registros de validación, informes de estado, *sign-off* de stakeholders, cuestionarios de acuerdo y mecanismos de retroalimentación como demostraciones o discusiones estructuradas [30]. Sin embargo, estos artefactos tampoco garantizan por sí mismos que el entendimiento compartido se haya logrado: un *sign-off* puede indicar aceptación formal sin demostrar necesariamente que todos los stakeholders interpretaron el requisito de la misma manera.

En conjunto, la evidencia sugiere que el entendimiento compartido se vuelve especialmente necesario en tres tipos de situaciones: cuando el significado del requisito aún se está construyendo, cuando ese significado debe formalizarse en artefactos, y cuando debe verificarse posteriormente contra el producto o los criterios de aceptación. Por tanto, las tareas, artefactos y situaciones identificados en elicitación, especificación y validación no representan momentos aislados, sino puntos de continuidad dentro del proceso de requisitos, en los que el entendimiento compartido puede favorecerse mediante artefactos, interacciones y actividades de revisión, sin que la literatura ofrezca una guía integrada sobre en qué secuencia aplicarlos, qué roles deben participar en cada momento, o cómo verificar que dicho entendimiento se logró efectivamente.

RQ3 — Momentos críticos

La literatura revisada muestra que los momentos más críticos para lograr el entendimiento compartido en IR corresponden a puntos del proceso donde el significado del requisito todavía es inestable y debe negociarse entre distintos stakeholders [27]. Esta criticidad también aparece cuando las necesidades obtenidas durante la elicitación deben transformarse en artefactos más formales, ya que esa transformación puede introducir ambigüedades, omisiones o interpretaciones divergentes [33]. De forma similar, el momento

de contrastar los requisitos frente a criterios de aceptación, pruebas o decisiones de implementación es crítico porque en él se verifica si el significado acordado se mantiene durante la validación [30]. A diferencia de la RQ2, que identifica las tareas, artefactos y situaciones donde aparece la necesidad de construir entendimiento compartido, esta pregunta se centra en los momentos de mayor riesgo, es decir, aquellos puntos donde la falta de alineación puede generar malentendidos, ambigüedades, pérdida de contexto o decisiones inconsistentes que se propagan a fases posteriores del proceso.

En la elicitación, uno de los momentos más críticos ocurre al inicio del proceso, cuando se identifican y priorizan los stakeholders que participarán en la definición de requisitos. Este momento es crítico porque una selección inadecuada de participantes puede introducir, desde el inicio, una visión incompleta o sesgada del sistema, dejando fuera necesidades, restricciones o expectativas relevantes del dominio [29]. Otro momento crítico en la elicitación aparece durante entrevistas, reuniones o *workshops*, cuando los participantes interactúan en tiempo real para expresar necesidades, aclarar dudas, negociar prioridades y corregir malentendidos; en estos espacios, el requisito aún no se ha estabilizado y su significado depende de explicaciones, ejemplos, preguntas y reformulaciones entre participantes [25]. También es crítico el momento en que se define el espacio del problema, se establece el vocabulario del dominio y se decide qué cuenta realmente como una necesidad válida, pues constituye la base semántica sobre la cual se escribirán, modelarán y validarán los requisitos posteriores [35]; de forma similar, el *handoff* entre quienes recogen las necesidades y quienes las traducen en trabajo técnico se vuelve un punto frágil, porque el significado acordado puede degradarse si no se preserva el contexto, la razón de ser y las decisiones tomadas durante la elicitación [37].

En la especificación, el momento más crítico ocurre cuando la información obtenida durante la elicitación debe transformarse en artefactos concretos, como historias de usuario, especificaciones en lenguaje natural, *feature files*, modelos, criterios de aceptación o plantillas estructuradas [24]. Este momento es crítico porque el significado deja de apoyarse únicamente en la conversación entre stakeholders y pasa a codificarse en documentos o representaciones que otras personas deberán interpretar posteriormente [33]. La criticidad aumenta cuando los requisitos se escriben en lenguaje natural, cuando cualidades vagas se transforman en criterios verificables, o cuando narrativas informales se convierten en modelos y estructuras más precisas [30]. Otro momento especialmente delicado en la especificación ocurre cuando se construye la visión global del sistema o se traduce un catálogo inicial de requisitos en artefactos accionables, ya que allí debe mantenerse la coherencia entre la necesidad original, su descomposición en requisitos más específicos y los artefactos derivados [26].

En la validación, los momentos más críticos aparecen cuando los stakeholders deben confirmar que lo especificado corresponde efectivamente a lo que se quería expresar, y cuando se define la evidencia, prueba o criterio con el que se comprobará su cumplimiento [38]. Este punto es crítico porque allí se confrontan la expectativa original, el requisito documentado, los criterios de aceptación, las pruebas y, en algunos casos, la implementación; si estos elementos no mantienen el mismo significado, pueden aparecer discrepancias tardías entre lo que los stakeholders esperaban, lo que se especificó y lo que finalmente se construyó [30]. La validación también se vuelve crítica durante las revisiones conjuntas de documentos, el *sign-off* final, los *sprint reviews*, la ejecución de *feature files* y la verificación de cumplimiento entre requisitos y producto [42]. Además, algunos estudios destacan como crítico el momento posterior de transferencia hacia desarrolladores, evaluadores u otros actores que no participaron en la discusión inicial de requisitos, ya que quienes reciben el requisito pueden acceder al artefacto documentado, pero no necesariamente al contexto, las razones y los matices que le dieron origen [43].

En conjunto, la literatura sugiere que los momentos más críticos son aquellos en los que se decide qué cuenta como un requisito válido, cuando ese significado se formaliza en artefactos, y cuando su cumplimiento debe demostrarse posteriormente a otros actores o frente al producto construido [39]. Estos momentos concentran tres riesgos principales: que los stakeholders no logren negociar una interpretación común; que el significado acordado se pierda o transforme al documentarse; y que las desalineaciones solo se vuelvan visibles durante la validación o la implementación [44].

RQ4 — Mecanismos, técnicas y enfoques

La literatura revisada muestra que los mecanismos, técnicas y enfoques asociados al entendimiento compartido en IR cumplen principalmente tres funciones: apoyar su construcción inicial, facilitar su formalización en artefactos, y contribuir a su mantenimiento cuando los requisitos cambian, se refinan o deben validarse. En términos generales, la literatura reporta mecanismos sociales, basados en la interacción directa entre stakeholders (entrevistas, reuniones, *workshops*, discusiones guiadas); mecanismos técnicos, apoyados en artefactos, modelos, plantillas o repositorios; y mecanismos sociotécnicos, que combinan ambos elementos al articular la conversación entre participantes con artefactos que hacen visible, estable y revisable lo acordado [27]. No obstante, estos mecanismos deben entenderse como apoyos parciales y situados, y no como una guía completa ni como evidencia concluyente de que el entendimiento compartido fue efectivamente alcanzado.

Durante la elicitación, la literatura reporta *workshops* colaborativos, entrevistas, discusiones guiadas, sesiones de co-creación y técnicas de negociación entre stakeholders como

mecanismos frecuentes [40]. Estos mecanismos apoyan principalmente la construcción inicial del entendimiento compartido, pues permiten que usuarios, clientes, analistas y desarrolladores expresen sus necesidades, hagan preguntas, aclaren expectativas y contrasten distintas interpretaciones del problema. En esta fase también se proponen mecanismos para organizar la participación y reducir sesgos durante las reuniones de requisitos: el uso de un moderador permite guiar la discusión, el *scribe* registra acuerdos y decisiones, y la pizarra compartida permite representar visualmente conceptos y relaciones que el grupo construye durante la sesión [25]. En contextos multilingües o distribuidos se reporta el uso de traducción automática en tiempo real y prácticas de *grounding* —preguntas, aclaraciones, confirmaciones y reformulaciones mediante las cuales los participantes verifican si realmente están entendiendo lo mismo— [25]. Otros mecanismos usados en la elicitación incluyen *story cards*, modelado participativo y software de comodelado, que permiten a los stakeholders representar conjuntamente procesos, escenarios o elementos del dominio [45]. El uso de *personas* también se reporta como mecanismo para representar características, necesidades y expectativas de los usuarios [21]; asimismo, los glosarios de dominio, como el *Language Extended Lexicon* (LEL), permiten acordar significados comunes para los términos relevantes del dominio, reduciendo el riesgo de que distintos stakeholders usen las mismas palabras con significados diferentes o términos distintos para referirse a la misma idea [35].

En la especificación, los mecanismos se orientan principalmente a formalizar y estabilizar el entendimiento alcanzado durante la elicitación. La literatura destaca el uso de plantillas estructuradas, formatos estandarizados y guías de escritura, que hacen explícitos los elementos mínimos de cada requisito y facilitan su revisión por distintos stakeholders [27]. Dentro de estos mecanismos se reportan propuestas orientadas a mejorar la expresión de requisitos en lenguaje natural, como *Structure for Unambiguous Requirement Expression* (SURE) [46], así como el uso de *feature files* de *Behavior-Driven Development* (BDD) estructurados mediante escenarios *Given/When/Then*, que permiten conectar la intención del requisito con comportamientos esperados y condiciones verificables [24]. También se identifican mecanismos basados en el modelado y la transformación de requisitos, como el modelado participativo, que permite construir representaciones del dominio con participación activa de los stakeholders [45], y el uso de *Action Sheets* como apoyo para estructurar información de requisitos en contextos de trabajo colaborativo [37]. Además, se reportan mecanismos de procesamiento de lenguaje natural (NLP) para convertir expresiones informales o poco estructuradas en requisitos más comprensibles y consistentes [47], y prácticas de trazabilidad y gestión explícita del cambio que permiten rastrear cómo cambia un requisito, quién lo modifica y qué artefactos se ven afectados [18]. También se reportan

mecanismos de control de calidad del lenguaje, como inspecciones focalizadas y guías de reescritura para detectar ambigüedades persistentes [33], así como propuestas de detección automatizada de *requirement smells* mediante expresiones regulares, NLP o aprendizaje automático [38].

En la validación, los mecanismos se orientan a verificar si el significado previamente construido y formalizado se mantiene cuando los stakeholders contrastan los requisitos con el producto, las pruebas o los criterios de aceptación. La literatura reporta el uso de criterios de aceptación, registros de validación y *sign-off* de stakeholders como mecanismos para formalizar la revisión y aceptación de requisitos [30]; también se identifica la trazabilidad entre necesidades, requisitos, código y pruebas, así como los *sprint reviews* y las demostraciones, como espacios de retroalimentación para revisar si lo especificado corresponde a lo construido [42]. En enfoques ágiles o distribuidos se enfatiza además el valor de la retroalimentación frecuente, las demostraciones, las revisiones de código y las reuniones *cross-functional* como mecanismos para aclarar y reajustar el entendimiento compartido cuando surgen cambios o desalineaciones [44]. Algunos estudios proponen mecanismos más ricos en contexto, como videos de *workshops* de requisitos, para comunicar a desarrolladores ausentes no solo el contenido acordado, sino también las razones, prioridades y matices de la interacción original [43]; en otros casos se propone la transparencia y la trazabilidad como mecanismos centrales para alinear roles con perspectivas distintas, por ejemplo entre perfiles legales, técnicos y de auditoría [48].

En conjunto, la evidencia sugiere que los mecanismos más robustos para apoyar el entendimiento compartido son aquellos que combinan interacción entre stakeholders, artefactos estructurados, trazabilidad y retroalimentación continua [3]. Los mecanismos de elicitación contribuyen principalmente a construir una base común inicial; los de especificación permiten formalizar y estabilizar el significado de los requisitos; y los de validación ayudan a verificar, mantener y reajustar ese significado frente al producto, las pruebas y los cambios del proceso. Sin embargo, la literatura revisada no ofrece un procedimiento integrado, paso a paso, que indique cómo seleccionar, combinar y aplicar secuencialmente estos mecanismos para garantizar el entendimiento compartido a lo largo del ciclo de requisitos.

RQ5 — Métricas, indicadores e instrumentos

La literatura revisada muestra que no existe una métrica única, directa y ampliamente aceptada para medir el entendimiento compartido como constructo. En su lugar, los estudios primarios reportan un conjunto de indicadores indirectos, instrumentos de evaluación y evidencia observable que permiten aproximar la medición de su logro en distintos

momentos del ciclo de requisitos [27]. Estos indicadores no siempre se presentan como métricas formalmente validadas; en algunos casos se usan de forma empírica dentro de los estudios reportados, mientras que en otros se proponen como señales o criterios útiles para evaluar el grado de alineación entre participantes. En general, la evidencia sugiere que el entendimiento compartido puede evaluarse a partir de señales de claridad, acuerdo, consistencia, trazabilidad, participación, estabilidad de los requisitos y capacidad de validar o implementar lo acordado [46]. Estas mediciones suelen distribuirse en tres momentos de aplicación: *antes*, para establecer una línea base o verificar condiciones iniciales; *durante*, para detectar rupturas de alineación mientras se construyen acuerdos o artefactos; y *después*, para evaluar si el entendimiento logrado se sostuvo hasta la validación, la implementación o la aceptación final del producto [42].

En la elicitación, la literatura propone principalmente indicadores vinculados a la interacción, la cobertura de perspectivas y la calidad del acuerdo alcanzado. En una fase *previa*, algunos estudios recomiendan establecer líneas base mediante instrumentos que permitan comprender condiciones previas que puedan afectar la comunicación; por ejemplo, en contextos multilingües se pueden aplicar pruebas de dominio del idioma para interpretar después si las dificultades de comunicación se relacionan con barreras lingüísticas [25]. También se reportan métricas de cobertura y participación, como el número de clases de stakeholders involucradas o la diversidad de participantes presentes en la sesión [42]. *Durante* la elicitación, la evaluación se centra en observar cómo interactúan los participantes y si logran construir acuerdos durante la conversación, mediante indicadores como solicitudes de aclaración, tipos de malentendidos, número de intervenciones por participante y señales de consenso o desacuerdo observadas en *workshops* o reuniones de requisitos [25].

En la especificación, la evaluación del logro del entendimiento compartido se centra principalmente en la calidad, estabilidad y comprensibilidad de los artefactos producidos. La literatura reporta métricas como el número de requisitos ambiguos, el número de interpretaciones distintas por stakeholder, la comprensibilidad del requisito con una única explicación, el número de conflictos detectados, la trazabilidad hacia una fuente específica y la frecuencia de actualizaciones o *change requests* sobre el documento [38]. También se reportan instrumentos como registros de validación, matrices de trazabilidad, bitácoras de *open issues*, escalas de utilidad percibida y evaluaciones de expertos sobre claridad, completitud estructural y reducción de la ambigüedad en la redacción [30]. En varios estudios, la aprobación del *feature file* a nivel de sistema, la calidad de los diagramas o modelos generados y la estabilidad de las historias de usuario o los escenarios BDD funcionan como evidencia indirecta de que el significado del requisito se mantiene suficientemente alineado [24].

En la validación, la literatura recomienda aplicar indicadores que permitan contrastar si lo especificado se entiende y verifica con un criterio común. Con este fin, en una fase *posterior* o de cierre, se reportan instrumentos como cuestionarios de satisfacción, involucramiento, facilidad para llegar a acuerdos y percepción de cumplimiento de necesidades tras la entrega del sistema [25]. También se usan como evidencia objetiva de la consolidación del entendimiento compartido el *sign-off* de stakeholders, el número de firmas de aprobación, la evaluación de criterios de aceptación en *sprint reviews*, el *compliance* entre requisitos y producto, y la trazabilidad entre necesidades, requisitos, código y pruebas [30]. Otros estudios sugieren usar el número de defectos, el retrabajo de pruebas, las inconsistencias entre requisito, implementación y prueba, o la latencia del ciclo de retroalimentación como señales posteriores e indirectas de desalineación [38]. En propuestas basadas en BDD, los informes de ejecución de *feature files* constituyen evidencia del grado en que el comportamiento esperado fue correctamente entendido, implementado y validado [24].

La literatura sugiere que los momentos de aplicación recomendados dependen del tipo de indicador y del propósito de la evaluación: los instrumentos *previos* permiten identificar condiciones iniciales que favorecen o limitan el entendimiento compartido; los instrumentos *durante* permiten monitorear su construcción mientras los stakeholders interactúan, aclaran conceptos, negocian significados y producen artefactos; y los instrumentos *posteriores* permiten verificar si el entendimiento alcanzado se mantuvo hasta la aprobación, implementación, prueba o aceptación del producto [42]. En conjunto, la evidencia muestra que la evaluación del entendimiento compartido depende menos de una escala única y más de una combinación de métricas de proceso, calidad documental, interacción social y resultados de validación [43].

2.4.11. Amenazas a la validez

Como en toda revisión sistemática, el proceso desarrollado está sujeto a amenazas que deben hacerse explícitas para interpretar correctamente sus resultados.

Sesgo de la estrategia de búsqueda. La cadena de búsqueda pudo no capturar todas las variantes terminológicas del fenómeno estudiado, dado que el entendimiento compartido no siempre se nombra explícitamente en la literatura, sino que se asocia a conceptos relacionados como comunicación, alineación, coordinación o colaboración. Para reducir este riesgo, la búsqueda se estructuró alrededor de los conceptos centrales del estudio e incluyó expresiones relacionadas como *Shared Understanding*, *Common Understanding* y *Cognitive Alignment*; además, los criterios de inclusión permitieron considerar estudios que abordaran fenómenos equivalentes dentro de las actividades de IR.

Sesgo de fuentes y filtros. La búsqueda se limitó a ScienceDirect, Scopus y Web of Science, y se restringió a artículos de investigación en inglés, de acceso abierto y publicados entre 2015 y 2026. Estas decisiones permitieron delimitar un corpus consistente con el alcance de la revisión, pero pudieron excluir estudios relevantes de otras fuentes, idiomas o tipos de publicación. Para controlar esta amenaza, se seleccionaron bases de datos reconocidas en Ingeniería de Software y se reportaron explícitamente los filtros aplicados.

Sesgo de selección de estudios. La aplicación de los criterios de inclusión y exclusión sobre títulos, resúmenes, palabras clave y textos completos implicó juicios de interpretación por parte de los revisores, especialmente en estudios donde el entendimiento compartido aparecía de forma indirecta. Para reducir este riesgo, los criterios se definieron a priori y el proceso de selección se realizó en dos niveles —título/resumen y texto completo—, con decisiones documentadas y discutidas entre los autores cuando surgían dudas sobre la pertinencia de un estudio (Sección 2.4).

Sesgo de extracción y síntesis de datos. La extracción de información y la síntesis temática pudieron estar sujetas a errores u omisiones debido a la heterogeneidad conceptual del tema. Para reducir esta amenaza, la extracción se guio por las categorías derivadas de las preguntas de investigación RQ1–RQ5 (Tabla 2.1), y los datos extraídos fueron revisados y contrastados entre los autores para reducir omisiones y mantener coherencia entre la evidencia recuperada y las categorías de análisis.

Sesgo de publicación y de constructo. Al centrarse en estudios primarios revisados por pares, la revisión pudo dejar fuera literatura gris, informes técnicos o experiencias industriales no publicadas. Además, dado que el entendimiento compartido es un constructo disperso, algunos estudios relevantes pudieron no usar esta terminología de forma explícita. Para controlar parcialmente esta limitación, se consideraron conceptos relacionados siempre que estuvieran vinculados a la elicitación, especificación o validación de requisitos y aportaran evidencia útil para responder las preguntas de investigación.

2.4.12. Síntesis de hallazgos

La síntesis temática organizada por preguntas de investigación arrojó cinco hallazgos principales que justifican las decisiones de diseño de THUNDERS-Master. Primero, el entendimiento compartido requiere representación adecuada de stakeholders y fuentes de conocimiento. Segundo, el lenguaje del dominio debe gestionarse de forma explícita para evitar que términos comunes oculten significados distintos. Tercero, las necesidades deben registrarse con fuente, contexto, valor esperado y supuestos. Cuarto, la trazabilidad es un mecanismo semántico —no solo administrativo— que conecta necesidad, requisito, criterio, decisión y evidencia de validación. Quinto, la validación debe revisar significado y

criterios de aceptación, no solo completitud formal. La RSL también muestra que no existe una definición operacional consolidada de entendimiento compartido para IR, que pocos estudios ofrecen una guía sistemática para construirlo y mantenerlo, y que su evaluación se apoya en indicadores indirectos más que en métricas o instrumentos validados.

Tabla 2.2: Hallazgos de la RSL y su implicación para THUNDERS-Master.

Hallazgo de la RSL	Implicación para THUNDERS-Master
El entendimiento compartido se describe mediante conceptos dispersos (alineación, comunicación, colaboración, modelos mentales).	Adoptar una definición operacional y traducirla en chequeos ligeros de alineación de interpretación.
Las rupturas aparecen cuando los stakeholders relevantes no participan o su autoridad/conocimiento no es claro.	Incluir un mapa mínimo de stakeholders con fuente de conocimiento, disponibilidad, representatividad y poder de decisión.
La ambigüedad y los vocabularios heterogéneos afectan elicitación y especificación.	Mantener un glosario vivo y una actividad explícita para resolver términos críticos o asignar responsable de aclaración.
La transformación de necesidades en requisitos puede perder contexto, valor o supuestos.	Cada necesidad conserva fuente, contexto y estado; cada requisito crítico se enlaza con su necesidad de origen.
La validación puede volverse superficial al enfocarse solo en la forma textual.	La validación cubre la cadena necesidad–requisito–criterio–significado y registra las discrepancias.
La literatura reporta mecanismos, pero no siempre una ruta integrada de aplicación.	THUNDERS-Master organiza estos mecanismos en tres fases y once actividades operativas.

2.4.13. Brecha de investigación

La evidencia sintetizada permite delimitar con precisión el problema que los enfoques actuales no resuelven. Primero, el entendimiento compartido se aborda de forma fragmentada: los estudios se concentran en artefactos, modelos o dinámicas sociales por separado, sin integrarlos en una ruta continua que cubra elicitación, especificación y validación. Segundo, no existe una definición operacional consolidada del entendimiento compartido para IR, lo que impide tratarlo como criterio verificable durante el trabajo. Tercero, aunque la literatura reporta múltiples mecanismos, no ofrece un procedimiento integrado, ligero y trazable que indique cómo seleccionarlos y combinarlos. Cuarto, su evaluación se apoya en indicadores indirectos que no distinguen entre existencia de documentación, aceptación formal y entendimiento real. Los marcos de desarrollo ampliamente adoptados atienden la coordinación mediante ceremonias y refinamiento, pero no prescriben cómo construir,

monitorear y validar el significado compartido de los requisitos ni qué evidencia mínima lo hace comprobable.

Esta brecha justifica metodológicamente la decisión de adaptar THUNDERS en lugar de diseñar un proceso nuevo. THUNDERS fue construido con Ingeniería de Métodos Situacionales y validado como un proceso completo y útil para construir, monitorear y asistir el entendimiento compartido, pero también extenso y cognitivamente demandante [7, 8]. Existe, por tanto, un proceso con la intención correcta pero con una complejidad inadecuada para las sesiones breves de la IR. La respuesta razonada no es partir de cero, sino aplicar una adaptación situacional que conserve la intención de THUNDERS y reduzca su carga, orientándola a las tres actividades del alcance. La Tabla 2.3 resume este razonamiento, que el Capítulo 4 concreta en decisiones de adaptación específicas.

Tabla 2.3: De la brecha identificada a la justificación de adaptar THUNDERS.

Brecha identificada en la RSL	Por qué THUNDERS es una base adecuada	Por qué requiere adaptación a IR
No hay una ruta integrada, ligera y trazable para construir, monitorear y validar el entendimiento compartido.	THUNDERS integra fases, tareas, roles, productos y mecanismos de monitoreo y asistencia del entendimiento compartido.	Su flujo completo es extenso para talleres, entrevistas y refinamientos; debe simplificarse conservando la intención.
Falta una definición operacional del entendimiento compartido verificable durante el trabajo.	THUNDERS trata el entendimiento compartido como objeto explícito de construcción y medición.	La operacionalización debe traducirse en chequeos ligeros y evidencia mínima propios de la IR.
Los mecanismos existen dispersos, sin criterios para seleccionarlos y combinarlos.	THUNDERS ya organiza mecanismos sociales, técnicos y sociotécnicos en un proceso coherente.	La selección debe regirse por su aporte directo al entendimiento compartido en elicitación, especificación y validación.
La evaluación se apoya en indicadores indirectos que confunden documentación con entendimiento.	THUNDERS incorpora momentos de validación y medición del entendimiento.	La evaluación de desempeño individual se retira del núcleo y se prioriza la validación semántica de requisitos.

Capítulo 3

Caracterización de las actividades de ingeniería de requisitos

3.1. Generalidades

Este capítulo caracteriza las tres actividades de IR que sirven de base para THUNDERS-Master: elicitación, especificación y validación. La caracterización toma como base principal la norma ISO/IEC/IEEE 29148:2018 [1] y se complementa con normas de ciclo de vida [49, 50], el modelo de calidad ISO/IEC 25010 [51], normas de usabilidad [52] y literatura académica. Su propósito es definir qué debe hacer el proceso, qué productos debe generar y qué evidencias debe conservar para apoyar el entendimiento compartido. Además de la norma, la caracterización se ancla en la evidencia de la revisión sistemática (Sección 2.4): cada actividad se lee no solo desde su definición normativa, sino desde las rupturas de entendimiento compartido, los momentos críticos y los mecanismos que la RSL identificó para ella. La Tabla 3.1 resume la lectura operativa adoptada.

3.2. Elicitación de requisitos

La elicitación es la actividad colaborativa, iterativa y orientada al contexto mediante la cual se identifican, contrastan, priorizan y refinan necesidades, expectativas, restricciones, escenarios, criterios de calidad y supuestos relevantes de los stakeholders [1]. No debe confundirse con la simple recolección de frases: su función es construir una comprensión inicial del problema, reconocer tensiones entre actores y transformar información dispersa en insumos útiles [2]. Coherente con la RSL, aquí se concentran las rupturas tempranas del entendimiento compartido —selección de stakeholders no representativos y vocabulario de

Tabla 3.1: Caracterización operativa de las actividades de ingeniería de requisitos.

Actividad	Propósito	Artefactos esperados	Relación con el entendimiento compartido
Elicitación	Descubrir, aclarar, contrastar y priorizar necesidades, restricciones, escenarios, supuestos y criterios de calidad.	Mapa de stakeholders; contexto de uso; escenarios; registro de necesidades; glosario inicial; conflictos y supuestos.	Construye la primera interpretación compartida del problema y hace visibles los desacuerdos.
Especificación	Transformar necesidades y acuerdos en requisitos bien formados, estructurados, trazables y verificables.	Requisitos individuales; atributos; taxonomía; matriz de evaluación; StRS, SyRS o SRS.	Formaliza el significado acordado, reduce la ambigüedad y preserva la trazabilidad.
Validación	Confirmar con stakeholders que los requisitos representan el sistema correcto y el contexto real de uso.	Actas de revisión; comentarios; decisiones; estado de aceptación; baseline versionada; registro de desacuerdos.	Transforma la interpretación documentada en acuerdo explícito o desacuerdo gestionado.

dominio ambiguo (RQ1, RQ3)—, por lo que la caracterización enfatiza la identificación de fuentes y la gestión explícita de la terminología.

3.2.1. Propósito

El propósito de la elicitación es hacer visible aquello que debe comprenderse antes de escribir requisitos: quiénes son los stakeholders, qué metas persiguen, qué restricciones enfrentan, cómo se usa o se usará el sistema, qué atributos de calidad importan y qué desacuerdos existen desde el inicio [3]. En THUNDERS-Master, esta actividad es clave porque el entendimiento compartido empieza antes de la documentación formal: si la elicitación deja supuestos ocultos o actores sin consultar, la especificación y la validación heredan vacíos difíciles de corregir [8].

3.2.2. Alcance y entradas

La elicitación cubre necesidades explícitas e implícitas, restricciones, criterios de aceptación, escenarios de operación y atributos de calidad, e incluye el reconocimiento de conflictos, variantes y prioridades iniciales. Debe considerar el contexto de uso —usuarios, tareas, ambientes, herramientas, objetivos y condiciones bajo las cuales el sistema tendrá sentido— [53], y consolidar necesidades de usuario a partir de varias fuentes, sin depender de una sola voz o técnica de levantamiento [54]. Sus entradas principales son el problema u oportunidad formulada, los objetivos de negocio, los conceptos operacionales preliminares, la información del dominio, el inventario inicial de stakeholders, la documentación existente y las restricciones regulatorias, técnicas y organizacionales.

3.2.3. Tareas, técnicas y productos

Las técnicas deben depender del tipo de fuente y de necesidad, no de una plantilla única; la elicitación moderna puede combinar interacción humana, análisis documental y datos generados por usuarios o sistemas, incluso apoyada en sistemas de recomendación [17, 55]. La Tabla 3.2 resume las tareas características y su evidencia esperada.

La selección de técnicas combina, según la fuente y la necesidad: entrevistas semiestructuradas; talleres facilitados; lluvia de ideas y sesiones de co-creación; observación del trabajo y *shadowing*; revisión de documentación técnica, normativa y operativa; escenarios, casos de uso, *user journeys* y *day-in-the-life*; prototipos de baja fidelidad y *mockups*; y análisis de *feedback* digital y fuentes de datos.

Tabla 3.2: Tareas características de la elicitación.

Tarea	Descripción operativa	Evidencia esperada
Preparar la elicitación	Definir alcance, estrategia, fuentes, técnicas y reglas.	Plan o agenda de elicitación.
Identificar stakeholders y fuentes	Reconocer usuarios, clientes, operadores, expertos, documentos y normas.	Mapa de stakeholders y fuentes.
Definir contexto de uso	Describir usuarios, tareas, ambientes y modos de operación.	Descripción de contexto y escenarios.
Elicitar necesidades explícitas e implícitas	Entrevistas, talleres, observación, prototipos, análisis del dominio.	Registro de necesidades y supuestos.
Analizar escenarios	Casos de uso, <i>user journeys</i> , <i>day-in-the-life</i> .	Escenarios revisados con stakeholders.
Priorizar y depurar	Ordenar necesidades, eliminar duplicados, detectar conflictos.	Priorización inicial y conflictos abiertos.
Transformar en requisitos de stakeholder	Convertir necesidades en enunciados con fuente y contexto.	Requisitos de stakeholder preliminares.
Retroalimentar y negociar	Devolver la interpretación para corregir y acordar significado.	Observaciones, acuerdos y cambios.

3.2.4. Roles involucrados

Participan el analista de requisitos o facilitador, representantes de usuarios finales, operadores del sistema, el cliente o patrocinador, expertos de dominio, perfiles de arquitectura, desarrollo, QA, UX o seguridad, y perfiles regulatorios o de cumplimiento cuando aplique. El facilitador no solo coordina reuniones: también ayuda a hacer visibles desacuerdos, términos ambiguos, supuestos no alineados y necesidades tácitas [7].

3.2.5. Artefactos y productos de trabajo

La elicitación produce el mapa de stakeholders, el inventario de fuentes de requisitos, la descripción del contexto de uso, los escenarios operacionales, el registro de necesidades, restricciones y criterios de calidad, un glosario inicial, el registro de preguntas abiertas, conflictos y supuestos, la *Stakeholder Requirements Specification* o insumo equivalente, y un repositorio de evidencias de origen que permita rastrear cada necesidad hasta su fuente.

3.2.6. Criterios de calidad y salida

La elicitación se considera suficientemente completa cuando hay cobertura razonable de stakeholders, el contexto de uso está descrito, las necesidades críticas están identificadas, los conflictos principales registrados y las necesidades priorizadas tienen fuente y

justificación [1]. Una mala elicitación se reconoce porque deja huecos de contexto, sesgos de representación, criterios de aceptación implícitos y atributos de calidad no verbalizados; estos problemas suelen convertirse después en ambigüedad, retrabajo o deuda de requisitos [5].

3.3. Especificación de requisitos

La especificación transforma la base de necesidades y acuerdos en un conjunto estructurado de requisitos bien formados, con atributos, trazabilidad y criterios de verificación o validación [1]. No es solo redacción: implica analizar el significado de los requisitos, clasificarlos, organizarlos, revisar su calidad y prepararlos para su uso por desarrollo, arquitectura, pruebas y stakeholders [56]. La RSL señala que en este punto el significado deja de apoyarse en la conversación y queda codificado para que otros lo interpreten (RQ3), y que la ambigüedad del lenguaje natural es la principal fuente de desalineación (RQ1); por ello la caracterización prioriza la calidad del enunciado y la trazabilidad hacia la necesidad de origen.

3.3.1. Propósito

El propósito de la especificación es convertir una visión orientada al problema y al uso en una representación documental rigurosa, que sirva como base de acuerdo, diseño, estimación, trazabilidad, verificación y validación [57]. La especificación reduce ambigüedad porque obliga a definir alcance, condiciones, restricciones, prioridades, criterios de calidad y relaciones entre requisitos [5].

3.3.2. Alcance y entradas

La especificación abarca cuatro dimensiones: la formulación del requisito individual, la calidad del conjunto, la estructuración documental y la preparación para la evaluación [1]. Los requisitos de calidad deben tratarse de forma explícita, porque en contextos ágiles suelen quedar incompletos, mal documentados o asumidos como conocimiento tácito [18]. Sus entradas principales son las necesidades y requisitos de stakeholder elicitados, el contexto de uso y los escenarios operacionales, las restricciones técnicas, regulatorias, organizacionales y de negocio, los criterios de calidad del producto o servicio, las decisiones previas de priorización y negociación, los supuestos y riesgos, y los sistemas legados, interfaces y documentación normativa aplicable. El modelo de calidad de ISO/IEC 25010 ayuda a

clasificar atributos como desempeño, compatibilidad, usabilidad, confiabilidad, seguridad, mantenibilidad y portabilidad [51].

3.3.3. Tareas que caracterizan la actividad

La Tabla 3.3 resume las tareas características de la especificación y su evidencia esperada.

Tabla 3.3: Tareas características de la especificación.

Tarea	Descripción operativa	Evidencia esperada
Definir frontera y alcance	Precisar qué sistema o elemento se describe, qué queda fuera y con qué interactúa.	Declaración de alcance y límites.
Definir estrategia de especificación	Establecer plantilla, niveles de abstracción, atributos obligatorios y responsabilidades.	Guía o plantilla de requisitos.
Formular requisitos	Redactar enunciados claros, singulares, factibles y verificables.	Requisitos individuales.
Clasificar requisitos	Separar funcionales, de calidad, interfaces, restricciones, datos, usabilidad y factores humanos.	Taxonomía o etiqueta por requisito.
Agregar atributos	Registrar identificador, fuente, prioridad, riesgo, <i>owner</i> , <i>rationale</i> , versión y método de evaluación.	Repositorio con atributos.
Analizar consistencia	Detectar contradicciones, duplicados, términos inconsistentes y vacíos de cobertura.	Lista de hallazgos de calidad.
Organizar documentación	Construir StRS, SyRS, SRS o artefacto equivalente según nivel y alcance.	Documento o repositorio estructurado.
Preparar evaluación	Asociar requisitos a criterios de aceptación, verificación o validación.	Matriz de evaluación o criterios asociados.

3.3.4. Reglas de calidad del requisito y del conjunto

Un requisito individual debe ser necesario, apropiado al nivel de abstracción, no ambiguo, completo, singular, factible, verificable, correcto y conforme al estilo adoptado. En la práctica, esto implica no mezclar varias obligaciones en una oración, evitar palabras abiertas como «rápido» o «adecuado» sin métrica, no incluir decisiones de diseño innecesarias y acompañar el enunciado con fuente, justificación y método de comprobación. El conjunto debe ser completo, consistente, factible, comprensible y validable, manteniendo trazabilidad hacia necesidades, fuentes, decisiones y criterios de aceptación [1]. Los atributos de calidad deben tratarse de forma explícita, pues en contextos ágiles suelen quedar incompletos o asumidos como conocimiento tácito [18].

3.3.5. Criterios de lenguaje y estilo

La especificación debe evitar superlativos, comparativos vagos, pronombres ambiguos, excepciones abiertas, condiciones incompletas y expresiones no verificables [1]. Para el proceso a adaptar, esto significa que la calidad de la especificación no puede depender solo de la habilidad individual de quien redacta: debe apoyarse en plantillas, glosarios, listas de chequeo y revisiones guiadas.

3.3.6. Roles involucrados

Participan el analista o ingeniero de requisitos, los stakeholders designados para revisar el significado, arquitectura y desarrollo, QA, UX, seguridad o cumplimiento, el responsable de configuración o repositorio, y el *product owner* o responsable de priorización, cuando exista.

3.3.7. Artefactos, productos de trabajo y criterios de salida

La especificación produce la *Stakeholder Requirements Specification*, la *System Requirements Specification*, la *Software Requirements Specification*, el glosario y las convenciones de redacción, el repositorio de requisitos con atributos, la matriz o enlaces de trazabilidad, las listas de chequeo de calidad y el registro de riesgos, prioridades, justificación y decisiones. Se considera lista para validación cuando cada requisito tiene identificador, fuente, justificación y trazabilidad mínima; cuando los requisitos críticos tienen criterio de evaluación; y cuando el conjunto no presenta contradicciones graves, duplicidades evidentes ni vacíos estructurales.

3.4. Validación de requisitos

La validación contrasta con evidencia si los requisitos representan adecuadamente las necesidades, restricciones, contexto de uso y criterios de éxito del sistema; su pregunta central es si se está definiendo el sistema correcto [1]. Se diferencia de la verificación: esta comprueba si el requisito está bien formulado, mientras que la validación comprueba si corresponde a lo que realmente se necesita. En procesos orientados al entendimiento compartido, la validación es crítica porque convierte la interpretación documentada en un acuerdo explícito o en un desacuerdo gestionado. La RSL advierte que la validación puede volverse superficial cuando se enfoca en la forma textual y no en la cadena necesidad-requisito-criterio (RQ4, RQ5); la caracterización adopta esa distinción como criterio de

salida.

3.4.1. Propósito

El propósito de la validación es reducir el riesgo de avanzar con requisitos formalmente correctos pero semánticamente equivocados: un requisito puede estar bien escrito y aun así no representar la intención del stakeholder o el contexto real de uso [1].

3.4.2. Alcance y entradas

La validación abarca la revisión del significado, la suficiencia del conjunto, la coherencia con escenarios reales, la resolución de conflictos, el acuerdo explícito y la consolidación de una *baseline* controlada [1]. Debe considerar usabilidad y calidad en uso, porque el valor de un requisito depende de usuarios, objetivos, tareas y contexto de interacción [52]. Sus entradas principales son la StRS, SyRS, SRS o conjunto equivalente de requisitos, los escenarios, conceptos operacionales y contexto de uso, las listas de chequeo de calidad, los criterios de aceptación, los prototipos, modelos, simulaciones o representaciones auxiliares, el registro de decisiones, riesgos, restricciones y justificaciones, y las versiones previas y solicitudes de cambio.

3.4.3. Tareas, técnicas y criterios de salida

La Tabla 3.4 resume las tareas características. Como técnicas se emplean revisiones con stakeholders clave, *checklists* de completitud, consistencia, viabilidad y testabilidad, prototipos de baja fidelidad, *mockups* navegables, diagramas de proceso, casos de uso y, en requisitos críticos, simulación o modelado formal. Los artefactos ligeros pueden mejorar la conversación entre stakeholders, especialmente cuando ayudan a representar atributos difíciles de discutir solo con texto [58]. La validación se considera cerrada cuando los stakeholders relevantes revisaron el conjunto en un formato comprensible, los defectos críticos están resueltos o registrados, existe evidencia de acuerdo o aceptación condicionada, y la nueva versión queda trazable y controlada.

3.4.4. Roles involucrados

Participan los stakeholders con capacidad de aceptar, rechazar o condicionar requisitos, el analista o facilitador de la revisión, el equipo técnico, QA, UX, seguridad o cumplimiento, el arquitecto de software, y el *product owner* o responsable de aceptación, cuando aplique.

Tabla 3.4: Tareas características de la validación.

Tarea	Descripción operativa	Evidencia esperada
Preparar la validación	Definir participantes, alcance, criterios y tipo de evidencia.	Plan o agenda de validación.
Retroalimentar requisitos	Devolver los requisitos para confirmar interpretación.	Comentarios y observaciones.
Ejecutar revisión estructurada	<i>Walkthroughs</i> , inspecciones, talleres o <i>checklists</i> .	Acta de revisión.
Usar artefactos de comprensión	Prototipos, modelos, escenarios o simulaciones.	Evidencia revisada con stakeholders.
Evaluar contra contexto	Revisar si sirven para usuarios, tareas y ambientes previstos.	Hallazgos por escenario.
Resolver conflictos	Negociar <i>trade-offs</i> de alcance, costo, riesgo y desempeño.	Decisiones y compromisos.
Obtener acuerdo	Registrar aprobación, rechazo o aceptación condicionada.	Estado de aceptación por requisito.
Fijar baseline y registrar cambios	Consolidar versión controlada y documentar ajustes.	Baseline versionada y registro de cambios.

3.4.5. Artefactos y productos de trabajo

La validación produce actas o reportes de revisión, la lista de observaciones y defectos de requisitos, los prototipos, modelos o escenarios revisados, el conjunto de requisitos aprobado, rechazado o condicionado, el registro de cambios, el registro de decisiones de negociación y la *baseline* del artefacto de requisitos. La comprensión colectiva del dominio y del producto es una condición para tomar decisiones acertadas sobre requisitos, especialmente en equipos interdisciplinarios [3].

3.5. Elementos transversales

Las tres actividades comparten elementos que sostienen el entendimiento compartido al pasar de una fase a otra (Tabla 3.5). La evidencia industrial muestra que una buena práctica de requisitos repercute en productividad, calidad, planificación, gestión de riesgos y coordinación con procesos posteriores [57].

Tabla 3.5: Elementos transversales a las tres actividades.

Elemento transversal	Función dentro del proceso
Gestión de stakeholders	Mantener visibles actores, responsabilidades, intereses y capacidad de decisión.
Contexto de uso	Anclar requisitos en usuarios, tareas, ambientes y objetivos reales.
Trazabilidad	Relacionar necesidades, requisitos, decisiones, cambios y criterios de evaluación.
<i>Rationale</i>	Explicar por qué se acepta, rechaza o modifica un requisito.
Calidad del lenguaje	Reducir ambigüedad y variación de interpretación.
Negociación	Gestionar conflictos entre stakeholders, restricciones y prioridades.
Iteración	Permitir ajustes sin perder control del avance.
Versionado y baseline	Proteger los acuerdos logrados y controlar cambios.
Evidencia de entendimiento	Registrar acuerdos, aclaraciones, conflictos, cobertura y retrabajo.

Capítulo 4

Ingeniería de métodos situacional y adaptación de THUNDERS

4.1. Generalidades

La adaptación de THUNDERS hacia THUNDERS-Master se fundamenta en la Ingeniería de Métodos Situacionales (SME). Este capítulo describe la metodología seguida para decidir el método de adaptación, analiza los elementos de THUNDERS relevantes para esa decisión, compara enfoques de adaptación de procesos, justifica el método seleccionado y presenta las decisiones de adaptación y su trazabilidad con THUNDERS.

4.2. Metodología de investigación

Este capítulo sigue una metodología documental y analítica orientada a seleccionar un método de adaptación de procesos para derivar THUNDERS-Master a partir de THUNDERS. Sus insumos principales son la monografía doctoral donde se especifica THUNDERS [7], el alcance definido para THUNDERS-Master (Capítulo 1) y la revisión sistemática sobre entendimiento compartido en Ingeniería de Requisitos desarrollada en este trabajo (Sección 2.4). Además, se revisan enfoques académicos de adaptación de procesos y métodos: Ingeniería de Métodos Situacionales, *method fragments*, *method chunks*, *software process tailoring*, líneas de procesos de software y adaptación ágil basada en contexto.

La estrategia de revisión no se presenta como una revisión sistemática exhaustiva independiente, sino como una revisión analítica orientada a la toma de decisión metodológica. Para mantener rigor, se usaron criterios de comparación explícitos y una lógica de síntesis argumentativa inspirada en guías de revisión de literatura en Ingeniería de Software [59],

consistente con la práctica de clasificar y mapear evidencia adoptada en la RSL de este trabajo [15]. La metodología seguida se resume en cinco pasos:

1. **Análisis del proceso original:** se identificaron propósito, fases, actividades, tareas, roles, productos de trabajo, mecanismos de monitoreo y limitaciones de THUNDERS con base en la monografía original [7].
2. **Caracterización de la necesidad de adaptación:** se analizó el alcance definido para THUNDERS-Master, en especial su foco en elicitación, especificación y validación de requisitos.
3. **Revisión de enfoques de adaptación:** se estudiaron métodos relacionados con Ingeniería de Métodos Situacionales, componentes reutilizables de método, *tailoring* de procesos, líneas de procesos y adaptación ágil basada en contexto.
4. **Comparación de alternativas:** se definieron criterios como capacidad de adaptar procesos existentes, compatibilidad con IR, compatibilidad con enfoques ágiles, reducción de complejidad, conservación de elementos esenciales, facilidad de aplicación en un trabajo de grado y respaldo académico.
5. **Propuesta de guía de adaptación:** se formuló una guía para transformar THUNDERS en THUNDERS-Master, incluyendo criterios para conservar, adaptar, fusionar o eliminar elementos del proceso original.

4.3. Análisis del proceso THUNDERS

THUNDERS fue creado para guiar actividades colaborativas de resolución de problemas mediante la construcción, el monitoreo y la asistencia del entendimiento compartido [7]; el proceso busca que los participantes comprendan qué deben hacer, cuál es el problema, cómo deben trabajar y cómo se validará la solución obtenida. Sus tres fases y su fundamento en SME ya se describieron en la Sección 2.2.8; este análisis se concentra en los elementos decisivos para la adaptación: los roles que distribuyen responsabilidades, los elementos que hacen visible el entendimiento compartido, y los factores que dificultan su aplicación en sesiones breves de requisitos.

THUNDERS define roles como *Activity coordinator*, *Activity designer*, *Instrument designer* e *Information collector*, además de productos de trabajo como guías, plantillas, listas de chequeo, mecanismos de validación, resultados individuales, resultados grupales y registros de evaluación [7]. Estos elementos son valiosos para una adaptación a requisitos porque permiten identificar responsabilidades, entradas, salidas y criterios de seguimiento; sin embargo, también pueden generar sobrecarga si se trasladan sin simplificación a contextos ágiles o livianos.

Los elementos de THUNDERS más relevantes para THUNDERS-Master son aquellos que hacen visible el entendimiento compartido: la definición explícita del problema, la socialización inicial, la construcción colaborativa de resultados, la validación de entendimiento individual y grupal, la validación de productos individuales y grupales, y la retroalimentación final [7]. Estos elementos permiten que el proceso no dependa únicamente de comunicación informal, sino que incluya puntos explícitos para comprobar si los participantes interpretan el problema y la solución de manera compatible; son, por tanto, los candidatos naturales a conservarse en el núcleo de THUNDERS-Master.

En contraste, la dificultad de aplicar THUNDERS completo se explica por su amplitud, la cantidad de tareas, el número de productos de trabajo, la presencia de múltiples roles y la necesidad de seguir mecanismos de validación detallados [7]. Estos aspectos son razonables para un proceso general de colaboración, pero resultan costosos en contextos de requisitos donde se necesitan ciclos cortos, artefactos livianos y respuesta rápida. La teoría de carga cognitiva respalda esta lectura: el diseño de una actividad puede imponer esfuerzo mental innecesario si introduce información, pasos o artefactos que no contribuyen directamente a la tarea principal [60], por lo que reducir la carga extrínseca implica conservar lo esencial y simplificar aquello que dificulta la ejecución sin aportar valor proporcional [61].

4.4. Revisión de métodos de adaptación de procesos

Se revisaron cinco enfoques de adaptación de procesos y métodos como candidatos para transformar THUNDERS en THUNDERS-Master.

4.4.1. Ingeniería de Métodos Situacionales

La Ingeniería de Métodos Situacionales, o *Situational Method Engineering* (SME), propone construir métodos específicos para una situación a partir de componentes reutilizables de métodos existentes [62]. La idea central es que los métodos universales rara vez encajan de forma completa en todos los contextos, por lo que resulta necesario ensamblar métodos adecuados a las características del proyecto, organización, equipo o dominio [63].

En términos generales, SME incluye actividades como especificar los requisitos del método, seleccionar componentes de método, ensamblar dichos componentes y validar el método resultante [64]. Este enfoque es especialmente aplicable a THUNDERS porque el propio proceso fue construido mediante Ingeniería de Métodos Situacionales [7]. Por tanto, adaptar THUNDERS mediante SME permite evolucionarlo usando un paradigma compatible con su construcción original.

La principal ventaja de SME es que permite conservar coherencia entre objetivos, actividades, roles, productos de trabajo y criterios de aplicación [65]. Su principal limitación es que puede volverse pesada si se exige construir repositorios extensos de componentes antes de generar una adaptación concreta [62]. En el caso de THUNDERS-Master, esta limitación se reduce porque THUNDERS ya está documentado en fases, actividades, tareas, roles y productos de trabajo [7].

4.4.2. Method fragments y method chunks

Los *method fragments* son partes reutilizables de métodos que pueden seleccionarse y combinarse para construir un método situacional [66]. Esta noción permite tratar un proceso no como un bloque indivisible, sino como una colección de piezas que pueden reutilizarse según una situación específica.

Los *method chunks* extienden esta idea al asociar cada fragmento con una intención, una situación de aplicación, una guía de uso y productos de trabajo relacionados [64]. Esta granularidad resulta útil para THUNDERS-Master porque permite transformar tareas o grupos de tareas de THUNDERS en unidades reutilizables orientadas a objetivos de Ingeniería de Requisitos. Por ejemplo, un chunk puede tener como intención “verificar entendimiento inicial del problema”, como situación “inicio de una sesión de elicitación”, como entrada “mapa de stakeholders” y como salida “registro de acuerdos y dudas”.

La principal ventaja de los chunks es que permiten una adaptación fina y trazable, conservando la intención de cada componente [67]. Su principal limitación es que requieren definir reglas de composición para evitar inconsistencias entre piezas seleccionadas [64]. En THUNDERS-Master, esta limitación se gestiona mediante una matriz que relaciona cada chunk con elicitación, especificación o validación (Tabla 4.4).

4.4.3. Software process tailoring

El *software process tailoring* se refiere a la adaptación de un proceso estándar u organizacional a las necesidades de un proyecto específico [68]. El SEI abordó tempranamente el *tailoring* como un mecanismo para ajustar procesos definidos en el marco del Software Capability Maturity Model [69]. Posteriormente, la literatura empírica mostró que el *tailoring* permite responder a condiciones particulares de proyecto, equipo y organización [68].

La revisión sistemática de Kalus y Kuhrmann identifica múltiples criterios usados para adaptar procesos de software y concluye que la adaptación depende fuertemente del contexto [70]. Esta conclusión es importante para THUNDERS-Master porque confirma que la decisión de conservar, adaptar o eliminar elementos de THUNDERS debe basarse en

criterios explícitos de contexto. Sin embargo, el *tailoring* por sí solo puede resultar insuficiente si no se complementa con un mecanismo que preserve la arquitectura conceptual del método original.

4.4.4. Software process lines

Las líneas de procesos de software, o *Software Process Lines* (SPrL), aplican principios de líneas de producto al dominio de procesos, con el objetivo de gestionar familias de procesos relacionados a partir de activos comunes y variabilidad explícita [71]. Este enfoque es útil cuando una organización necesita derivar múltiples variantes de un proceso para diferentes contextos, dominios o tipos de proyecto.

Para THUNDERS-Master, SPrL ofrece una visión interesante a largo plazo porque permitiría gestionar variantes para educación, salud o contextos empresariales, además de la variante de Ingeniería de Requisitos desarrollada en este trabajo. No obstante, para una primera adaptación de THUNDERS, SPrL puede introducir más infraestructura metodológica de la necesaria. Por esta razón, se considera más adecuada como perspectiva futura que como método principal de adaptación inicial.

4.4.5. Adaptación ágil basada en contexto

La adaptación ágil estudia cómo seleccionar, ajustar o combinar prácticas ágiles según las características del proyecto, el equipo y la organización [72]. Kruchten sostiene que la adopción de enfoques ágiles debe contextualizarse considerando factores como tamaño del equipo, criticidad, distribución, gobernanza, tasa de cambio y estabilidad arquitectónica [73].

Este enfoque es relevante para THUNDERS-Master porque la versión adaptada debe ser liviana, iterativa, compatible con retroalimentación frecuente y útil para equipos que no pueden asumir un proceso extenso. Sin embargo, la adaptación ágil se enfoca principalmente en seleccionar prácticas ágiles y no necesariamente en preservar la estructura semántica de un proceso existente. Por ello, su papel en THUNDERS-Master es complementario: aporta criterios de ligereza (Sección 5.2), pero no reemplaza el marco de adaptación principal.

4.5. Comparación de enfoques de adaptación

Se compararon cinco enfoques —SME orientada por chunks, method fragments/chunks como técnica aislada, *software process tailoring*, *software process lines* y adaptación ágil

basada en contexto— frente a criterios de adaptación de procesos existentes, compatibilidad con IR, compatibilidad ágil, reducción de complejidad, conservación de elementos esenciales y facilidad de aplicación en un trabajo de grado. La Tabla 4.1 resume el análisis cualitativo.

Tabla 4.1: Comparación de métodos de adaptación frente al caso THUNDERS-Master.

Método	Adapta procesos existentes	Compat. con IR	Compat. ágil	Reduce complejidad	Conserva elementos esenciales	Ref.
SME orientada por chunks	Alta: ensambla métodos situacionales desde componentes reutilizables.	Alta: puede orientar chunks a elicitación, especificación y validación.	Media-alta: si se combinan criterios de contexto y documentación mínima.	Alta: permite seleccionar solo los componentes necesarios.	Alta: cada chunk conserva intención, entradas y salidas.	[62]
Method fragments/chunks aislados	Alta: trabaja con unidades reutilizables de método.	Alta: cada unidad puede asociarse a una actividad de IR.	Media: necesita reglas adicionales de ligereza.	Alta, si se seleccionan chunks mínimos.	Media-alta: conserva la intención de los componentes, pero exige reglas de composición.	[64]
Software process tailoring	Alta: ajusta procesos estándar a proyectos concretos.	Media-alta: puede usar criterios de contexto de IR.	Alta: compatible con adaptación pragmática.	Media: no siempre define cómo reducir sin perder coherencia.	Media: puede eliminar elementos sin preservar la arquitectura conceptual.	[70]
Software process lines	Alta: gestiona familias de procesos con variabilidad explícita.	Media: requiere modelar una familia más amplia de variantes.	Media: su infraestructura puede resultar pesada para entornos livianos.	Media: reduce por derivación, pero exige modelado de variabilidad.	Alta: gestiona activos comunes entre variantes.	[71]
Adaptación ágil basada en contexto	Media: suele ajustar prácticas más que procesos completos.	Alta: la IR puede integrarse a prácticas ágiles de refinamiento y validación.	Alta: su propósito es contextualizar prácticas ágiles.	Alta: promueve suficiencia, retroalimentación rápida y bajo desperdicio.	Media: no garantiza conservar el núcleo conceptual de THUNDERS.	[72]

4.6. Método seleccionado

El método seleccionado es una *SME orientada por method chunks*, complementada con criterios de *tailoring* contextual [70] y reducción de carga cognitiva. La selección se justifica porque: THUNDERS fue construido con SME [7]; los chunks permiten una granularidad adecuada para seleccionar lo que aporta valor directo a IR; el *tailoring* aporta criterios para ajustar el proceso al contexto; y la teoría de carga cognitiva respalda reducir elementos

que aumentan el esfuerzo mental sin aportar valor proporcional. Frente a las alternativas de la Tabla 4.1, la SME orientada por chunks es la única que satisface a la vez los criterios decisivos para este trabajo: continuidad metodológica con la construcción original de THUNDERS, granularidad para conservar solo los componentes con aporte directo a la IR, preservación de la intención del proceso y viabilidad de aplicación en un trabajo de grado; el *software process tailoring* y las *software process lines* no preservan por sí solos la arquitectura conceptual del método, y la adaptación ágil basada en contexto no garantiza conservar su núcleo. La Tabla 4.2 propone una guía de aplicación de SME orientada por chunks para derivar THUNDERS-Master, basada en la lógica de especificar requisitos del método, seleccionar componentes, ensamblarlos y validarlos [64], e incorpora criterios de adaptación contextual identificados en la literatura de *software process tailoring* [70].

4.7. Decisiones de adaptación y trazabilidad con THUNDERS

La regla principal es conservar la intención de THUNDERS, no su complejidad completa: un componente solo pasa al núcleo operativo si aporta de forma directa al entendimiento compartido en IR y si su salida puede comprobarse mediante evidencia mínima. Así, la caracterización amplia de participantes se convierte en un mapa mínimo de stakeholders; el diseño completo de actividad se simplifica como canvas de sesión; la evaluación de desempeño de participantes se elimina del núcleo operativo; y la validación de solución se reinterpreta como validación semántica de la cadena necesidad–requisito–criterio.

4.7.1. Criterios de decisión de adaptación

Para que estas decisiones no dependan del juicio puntual de los autores, cada componente de THUNDERS se evaluó con un conjunto explícito y estable de criterios, anclados en la evidencia de la RSL (Sección 2.4) y en la caracterización de la IR (Capítulo 3). La Tabla 4.3 define esos criterios y su fundamento; la decisión sobre cada componente —conservar, adaptar, simplificar, fusionar, hacer opcional o eliminar— resulta de aplicarlos de forma conjunta.

Como regla operativa, cada componente puede valorarse de 1 a 5 en aporte al entendimiento compartido, utilidad para IR, costo, carga cognitiva, frecuencia y dependencia: se conserva cuando su aporte al entendimiento compartido y su utilidad para IR son altos; se adapta cuando su valor conceptual es alto pero su costo o carga cognitiva también lo son; se fusiona cuando es redundante con otro artefacto, rol o tarea; y se elimina cuan-

Tabla 4.2: Guía de aplicación de la SME orientada por chunks para derivar THUNDERS-Master.

Etapas	Actividad propuesta	Producto esperado
Caracterizar el contexto	Describir tipo de proyecto, dominio, criticidad, tamaño del equipo, disponibilidad de stakeholders, modalidad presencial o distribuida, nivel de agilidad y madurez en IR.	Perfil de contexto para aplicar THUNDERS-Master.
Analizar THUNDERS original	Descomponer THUNDERS en fases, actividades, tareas, roles, productos de trabajo, mecanismos de monitoreo, asistencia y validación.	Catálogo base de elementos de THUNDERS.
Identificar elementos esenciales	Determinar qué elementos de THUNDERS aportan directamente a construir, monitorear o validar entendimiento compartido.	Lista de elementos esenciales para THUNDERS-Master.
Identificar elementos opcionales	Determinar qué elementos son útiles solo en contextos complejos, distribuidos, críticos o con alto conflicto semántico.	Lista de elementos opcionales y condiciones de activación.
Convertir elementos en chunks	Representar tareas o grupos de tareas como chunks con intención, situación, entradas, salidas, roles y criterios de aplicación.	Repositorio inicial de chunks.
Mapear chunks a IR	Relacionar cada chunk con elicitación, especificación o validación; eliminar o transformar elementos sin relación clara con estas actividades.	Matriz chunk-actividad de IR (Tabla 4.4).
Aplicar criterios de simplificación	Evaluar cada chunk según su valor para el entendimiento compartido, utilidad en IR, costo, carga cognitiva, frecuencia, dependencia y redundancia.	Decisión de conservar, adaptar, fusionar o eliminar (Tabla 4.3).
Ensamblar THUNDERS-Master	Ordenar los chunks seleccionados en un flujo liviano para preparar, ejecutar, consolidar y validar actividades de requisitos.	Modelo conceptual operativo (Figura 5.1, Capítulo 5).
Validar la propuesta	Evaluar utilidad, facilidad de uso, carga cognitiva, completitud, claridad y capacidad de apoyar entendimiento compartido mediante expertos, piloto o caso de estudio.	Evidencia empírica y versión ajustada de THUNDERS-Master.

Tabla 4.3: Criterios de decisión para adaptar los componentes de THUNDERS.

Criterio	Pregunta que responde	Fundamento
Aporte directo al entendimiento compartido en IR	¿El componente contribuye a construir, monitorear o validar el significado compartido de los requisitos?	Hallazgos de la RSL (RQ1–RQ4).
Verificabilidad por evidencia mínima	¿Su salida puede comprobarse con un artefacto o evidencia observable y ligera?	Indicadores indirectos de la RSL (RQ5).
Costo de adopción y carga cognitiva	¿El esfuerzo que introduce es proporcional a su valor en una sesión breve?	Validación de THUNDERS como proceso extenso y demandante [8]; teoría de carga cognitiva [60, 61].
Pertinencia para elicitación, especificación y validación	¿El componente corresponde al alcance de las tres actividades caracterizadas?	Caracterización de la IR (Capítulo 3).
Valor informacional	¿El producto generado se usa para tomar decisiones, validar requisitos o mantener trazabilidad?	Evita artefactos que no se reutilizan en pasos posteriores del proceso.
Frecuencia de uso	¿El elemento se usará en la mayoría de sesiones de requisitos?	Los elementos de uso excepcional se activan solo por contexto (Tabla 5.4).
Compatibilidad ágil	¿El elemento es compatible con sesiones cortas, retroalimentación frecuente y documentación suficiente?	Encaje de THUNDERS-Master en marcos ágiles (Sección 5.2).
Dependencia metodológica	¿Eliminar el elemento rompe la coherencia de otros chunks esenciales?	Reglas de composición de <i>method chunks</i> [64].
Redundancia	¿El elemento duplica entradas, salidas o decisiones de otro elemento?	Evita fusionar información ya cubierta por otro artefacto o tarea.

do su aporte al entendimiento compartido y su utilidad para IR son bajos y su costo de aplicación es medio o alto.

La Tabla 4.4 complementa estos criterios relacionando cada elemento de THUNDERS con la actividad de IR a la que se transforma y con el artefacto mínimo que produce en THUNDERS-Master; el Capítulo 5 detalla cómo estos artefactos se integran en las actividades operativas A1–A11.

Tabla 4.4: Mapeo inicial de elementos de THUNDERS hacia actividades de IR en THUNDERS-Master.

Elemento de THUNDERS	Actividad de IR	Transformación propuesta	Artefacto mínimo
Caracterizar participantes y contexto	Elicitación	Reducir instrumentos extensos a una ficha breve de stakeholders, conocimiento del dominio, responsabilidades y restricciones.	Ficha de contexto y mapa de stakeholders.
Definir tema y problema de la actividad	Elicitación	Unificar descripción del problema, objetivo de la sesión y alcance inicial en una plantilla corta.	Enunciado compartido del problema.
Diseñar actividad colaborativa	Elicitación	Convertir el diseño completo de actividad en un guion de taller de requisitos.	Guion de sesión de elicitación.
Socializar el problema	Elicitación	Mantener la socialización inicial y agregar verificación explícita de vocabulario común.	Registro de alineación inicial.
Construir resultado grupal	Especificación	Transformar resultados colaborativos en historias, requisitos, escenarios o criterios de aceptación.	Backlog, ficha de requisito o escenario.
Validar entendimiento individual y grupal	Validación	Usar una lista breve de chequeo para confirmar interpretación común de requisitos y criterios de aceptación.	Checklist de entendimiento compartido.
Validar productos grupales	Validación	Revisar claridad, consistencia, trazabilidad y verificabilidad del artefacto de requisitos.	Acta de validación de requisitos.
Retroalimentar y cerrar	Validación	Cerrar con acuerdos, desacuerdos, decisiones pendientes y ajustes requeridos.	Registro de retroalimentación y decisiones.

La Tabla 4.5 aplica los criterios de decisión y resume la decisión adoptada para cada componente.

El resultado de aplicar estos criterios es un núcleo deliberadamente ligero, cuya baja carga es la condición que permite integrar THUNDERS-Master dentro de flujos ágiles sin introducir burocracia (Sección 5.2, Tabla 5.1).

Tabla 4.5: Trazabilidad resumida entre THUNDERS y THUNDERS-Master.

Componente THUNDERS	Decisión	Justificación
C-01 Definir la población	Adaptar	La caracterización completa es costosa; en IR basta con asegurar representación, conocimiento y autoridad de decisión.
C-02 Definir el tema de la actividad	Conservar/Adaptar	Representa el punto inicial de alineación entre problema, alcance y objetivo.
C-03 Diseñar la actividad	Simplificar/Fusionar	Un diseño completo es demasiado pesado para sesiones ágiles de requisitos.
C-04 Definir los grupos	Opcional (activable)	No siempre se requieren múltiples grupos; se prioriza participación representativa.
C-05 Diseño del material	Simplificar	Preparar material es útil, pero no debe duplicar documentación existente.
C-06 Diseñar validación y evaluación	Dividir	Separa la validación semántica de requisitos de la evaluación del proceso como investigación.
C-07 Formar los grupos	Eliminar/Fusionar	Duplica la definición de participación y aporta poco en sesiones pequeñas.
C-08 Describir la actividad	Conservar/Adaptar	Momento clave para confirmar el entendimiento inicial del problema.
C-09 Distribuir el material	Fusionar	No debe ser actividad independiente cuando los artefactos ya están disponibles.
C-10 Desarrollar la actividad colaborativa	Adaptar como núcleo	Se convierte en la actividad central, transformada al flujo real de IR.
C-11 Validar la solución	Adaptar como núcleo	Se enfoca en validar la correspondencia semántica, no una solución implementada.
C-12 Evaluar desempeño de participantes	Eliminar del núcleo	No mide directamente calidad de requisitos ni entendimiento compartido y aumenta la sobrecarga.
C-13 Cerrar la actividad	Conservar/Adaptar	Evita la pérdida de contexto y asegura que los acuerdos sigan siendo trazables.

Capítulo 5

THUNDERS-Master como proceso operativo

Estado del proceso

La especificación presentada en este capítulo corresponde a la **primera versión de THUNDERS-Master (v1.0)**: una especificación formal de trabajo derivada de la RSL, la caracterización de la IR, el análisis de THUNDERS y la Ingeniería de Métodos Situacionales. No es un proceso empíricamente validado. Está concebida para evolucionar de manera iterativa e incremental mediante un taller de validación con expertos y usuarios clave, y un pilotaje en un contexto real (agrícola). La retroalimentación obtenida producirá versiones posteriores (v1.1, v2.0, ...) cuyos cambios quedarán documentados. Por ello, fases, actividades, roles y artefactos deben leerse como una base estable pero abierta a refinamiento, no como un diseño definitivo.

5.1. Generalidades

THUNDERS-Master es un proceso ligero para apoyar la construcción, el monitoreo y la validación del entendimiento compartido durante la elicitación, especificación y validación de requisitos. No reemplaza Scrum, Kanban, Design Thinking, refinamiento de backlog ni otras prácticas existentes: las complementa con orientación explícita para hacer visible el significado común y dejar evidencia mínima que permita seguir trabajando sin perder contexto. La unidad de aplicación es una sesión, taller, refinamiento o revisión de requisitos. El proceso se estructura en tres fases heredadas de THUNDERS y reinterpretadas para IR —Beginning, Developing y Measuring/Closing— y se desarrolla mediante once actividades operativas (A1–A11) más una actividad opcional (A4b). La Figura 5.1 presenta el modelo

conceptual operativo.

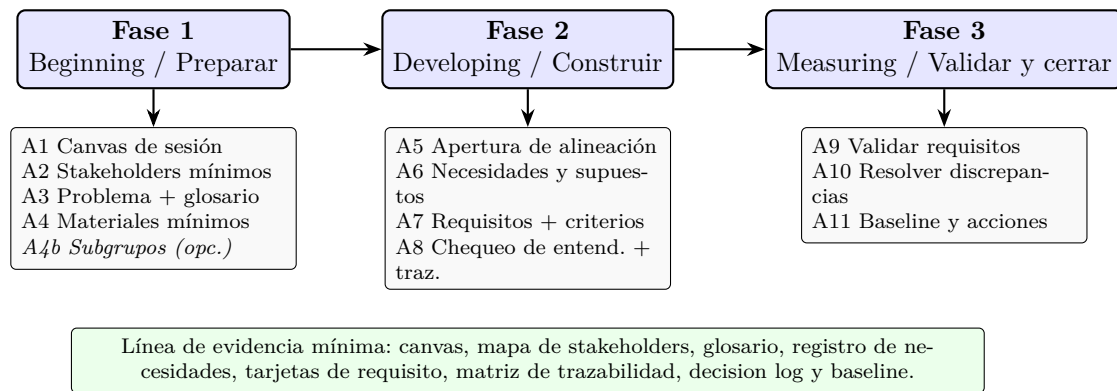


Figura 5.1: Modelo conceptual operativo de THUNDERS-Master.

El modelo expresa una idea clave: el entendimiento compartido suficiente no se asume por la existencia de documentos. Se considera construido cuando los participantes pueden reformular el problema de forma equivalente, los términos críticos tienen definición acordada o responsable, cada requisito crítico mantiene relación con una necesidad y un criterio, los desacuerdos quedan resueltos o asignados, y el cierre produce una versión identificable de los requisitos trabajados.

5.2. Encaje de THUNDERS-Master en marcos ágiles

El propósito de THUNDERS-Master es apoyar la construcción, el monitoreo y la validación del entendimiento compartido; por eso se concibe como una buena práctica que se aplica *dentro* de las metodologías ágiles, no como un marco que las reemplace. Marcos como Scrum o Kanban ya coordinan el trabajo mediante ceremonias, refinamiento y revisiones, pero no prescriben cómo asegurar que los participantes interpreten de la misma manera el problema, las necesidades y los criterios de aceptación. THUNDERS-Master cubre precisamente esa capa semántica y puede integrarse en los espacios ágiles existentes sin crear reuniones nuevas: sus actividades se acoplan a ceremonias que el equipo ya realiza (Tabla 5.1).

Este encaje también responde a la preocupación por una posible sobrecarga documental. Los artefactos de THUNDERS-Master no son entregables adicionales, sino campos mínimos que viven dentro de las herramientas que el equipo ya usa —backlog, wiki, *issues*, actas o documentos colaborativos (Sección 5.8)— y que solo se intensifican cuando el contexto lo exige (Tabla 5.4). Así, el proceso agrega trazabilidad y evidencia de entendimiento sin penalizar la agilidad: la documentación se limita a lo que permite verificar

quién originó un requisito, qué necesidad expresa y qué criterio demuestra que se cumple.

Tabla 5.1: Encaje de las actividades de THUNDERS-Master en ceremonias ágiles.

Actividades T-M	Espacio ágil de acople	Aporte al entendimiento compartido
A1–A4 (Preparar)	Preparación de <i>sprint planning</i> y refinamiento; <i>kickoff</i> de iniciativa.	Fija objetivo, stakeholders, problema y glosario antes de comprometer trabajo.
A5–A8 (Construir)	Refinamiento de <i>backlog</i> y escritura de historias/criterios.	Convierte necesidades en requisitos trazables y aplica micro-chequeos de interpretación.
A9–A11 (Validar y cerrar)	<i>Sprint review</i> , aceptación y <i>Definition of Done</i> .	Valida la cadena necesidad–requisito–criterio y fija una baseline acordada.

5.3. Requisitos del método

Los requisitos del método funcionan como contrato de diseño: permiten comprobar que el proceso no se limita a renombrar fases de THUNDERS, sino que responde a necesidades específicas de la IR. Se consolidan requisitos relacionados con stakeholders, contexto de sesión, problema compartido, lenguaje común, necesidades y supuestos, transformación a requisitos, calidad semántica, trazabilidad mínima, criterios de aceptación, decisiones y desacuerdos, chequeo de entendimiento, asistencia ligera, variabilidad contextual, evaluación externa y no transferencia excesiva. El proceso distingue elementos obligatorios (protegen el significado mínimo), recomendados (ayudan cuando la sesión es compleja) y opcionales (se activan por contexto: distribución, criticidad, regulación o baja madurez del equipo).

5.4. Fase 1: Beginning / Preparar

La fase *Beginning* prepara la sesión antes de elicitar o especificar. Su propósito es reducir incertidumbre inicial, delimitar el trabajo y asegurar que participen las fuentes correctas.

- **A1 Canvas de sesión.** Define objetivo, alcance, exclusiones, modalidad, agenda, técnica, insumos, salida esperada, regla de participación y criterio de cierre.
- **A2 Stakeholders mínimos.** Identifica actores, fuente de conocimiento, poder de decisión, disponibilidad, posibles conflictos y prioridad de participación.

- **A3 Problema y glosario semilla.** Formula una declaración compartida del problema y registra términos de dominio que pueden generar ambigüedad.
- **A4 Materiales y validación mínima.** Prepara insumos, preguntas guía, plantillas y criterios que se usarán durante la sesión.
- **A4b Subgrupos/turnos (opcional).** Se activa si hay muchos participantes, asimetrías de poder, distribución geográfica o necesidad de asegurar participación equilibrada.

5.5. Fase 2: Developing / Construir

La fase *Developing* es el núcleo del proceso. Su objetivo es construir requisitos de manera colaborativa mientras se monitorea la comprensión compartida.

- **A5 Apertura de alineación.** Revisa objetivo, alcance, reglas, participantes, problema compartido y términos críticos para partir de una base semántica común.
- **A6 Necesidades y supuestos.** Elicita necesidades, restricciones, supuestos, valor esperado y contexto de uso; cada necesidad debe tener fuente, contexto y estado.
- **A7 Requisitos/historias y criterios.** Transforma necesidades en requisitos, historias, escenarios o criterios de aceptación, conservando la relación con la necesidad fuente.
- **A8 Chequeo de entendimiento y trazabilidad.** Aplica *micro-chequeos* en puntos críticos: pedir reformulación, contrastar interpretación, revisar criterios observables y completar una matriz mínima de trazabilidad.

Los micro-chequeos son una decisión central: THUNDERS-Master no espera al final para descubrir discrepancias, sino que introduce revisiones breves durante la construcción (por ejemplo, pedir a dos participantes que reformulen un requisito, preguntar qué evidencia demostraría que un criterio se cumple, o marcar un supuesto como pendiente cuando falta información).

5.6. Fase 3: Measuring / Validar y cerrar

La fase *Measuring* se reinterpreta para IR: la evaluación de desempeño de participantes no pertenece al núcleo operativo. La fase se concentra en validar el significado de los requisitos, resolver discrepancias y cerrar una baseline.

- **A9 Validar requisitos y criterios.** Revisa la cadena necesidad–requisito–criterio–significado con los stakeholders. Un requisito no se considera listo solo por estar bien escrito: debe estar vinculado a una necesidad, contar con criterio observable y ser

interpretado coherentemente.

- **A10 Resolver discrepancias.** Registra diferencias de interpretación, conflictos de alcance, desacuerdos de prioridad o criterios no observables; cada discrepancia se resuelve o queda asignada con responsable y fecha.
- **A11 Baseline y acciones.** Cierra la sesión con una versión identificable de los requisitos, decision log, pendientes, responsables y próximos pasos.

La Tabla 5.2 sintetiza las once actividades con su entrada, salida y criterio de cierre.

5.7. Roles y responsabilidades

Los roles se reducen para evitar especialización innecesaria. En equipos pequeños, una persona puede asumir más de un rol, siempre que no se pierdan tres responsabilidades: facilitar la interacción, especificar requisitos y registrar evidencias. La Tabla 5.3 resume los roles simplificados.

5.8. Artefactos mínimos y evidencia observable

Los artefactos se diseñan como campos mínimos que pueden vivir dentro de herramientas existentes (backlog, wiki, issues, actas o documentos colaborativos): canvas de sesión, mapa de stakeholders, glosario vivo, registro de necesidades, tarjeta de requisito o historia, matriz mínima de trazabilidad, decision log y baseline. La evidencia mínima no busca llenar formatos por obligación, sino permitir verificación posterior: un revisor experto debería poder responder quién originó un requisito, qué necesidad expresa, qué criterio demuestra que se cumple, qué término crítico se acordó, qué desacuerdo quedó abierto, quién decide y qué versión fue validada.

5.9. Variabilidad contextual y criterios de activación

THUNDERS-Master no se ejecuta siempre con la misma intensidad. La variabilidad contextual define cuándo activar elementos adicionales, en línea con la adaptación estructurada de procesos [70] y evitando su aplicación rígida en contextos ágiles [72]. La Tabla 5.4 resume los criterios.

Tabla 5.2: Especificación resumida de las actividades de THUNDERS-Master.

Actividad	Propósito	Salida / evidencia mínima	Criterio de cierre
A1 Canvas	Definir objetivo, alcance, agenda y cierre.	Canvas breve de sesión.	La sesión tiene objetivo y salida esperada.
A2 Stakeholders	Asegurar fuentes de conocimiento y autoridad.	Mapa mínimo de stakeholders.	Los requisitos críticos tienen fuente o validador.
A3 Problema + glosario	Alinear el problema y la terminología crítica.	Enunciado del problema y glosario semilla.	Los participantes reformulan el problema de forma consistente.
A4 Materiales	Preparar insumos y plantillas.	Paquete mínimo de artefactos referenciados.	Los insumos están disponibles sin duplicar documentación.
A5 Apertura	Confirmar un punto de partida común.	Ajustes iniciales y reglas acordadas.	No quedan dudas críticas sobre el alcance.
A6 Necesidades	Elicitar necesidades con fuente y contexto.	Registro de necesidades y supuestos.	Cada necesidad tiene fuente, contexto y estado.
A7 Requisitos + criterios	Transformar necesidades en artefactos.	Requisitos o historias y criterios de aceptación.	Cada requisito se enlaza con su necesidad de origen.
A8 Chequeo + trazabilidad	Monitorear interpretación y trazabilidad.	Checklist y matriz mínima de trazabilidad.	Las discrepancias críticas están identificadas.
A9 Validar	Confirmar correspondencia y acuerdo.	Requisitos aceptados o corregidos.	Los criterios son observables y validados.
A10 Discrepancias	Gestionar diferencias de significado.	Decision log y acciones.	Todo desacuerdo crítico se resuelve o asigna.
A11 Baseline	Establecer baseline y continuidad.	Baseline, responsables y próximos pasos.	Existe una versión identificable de lo trabajado.

Tabla 5.3: Roles simplificados en THUNDERS-Master.

Rol	Responsabilidad principal	Condición de uso
Facilitador de requisitos	Conducir la sesión, gestionar participación, aplicar micro-chequeos y cerrar.	Obligatorio.
Analista / especificador	Transformar necesidades en requisitos, historias, escenarios y criterios.	Obligatorio.
Relator / gestor de trazabilidad	Registrar términos, necesidades, decisiones, desacuerdos, enlaces y baseline.	Recomendado; obligatorio en sesiones grandes.
Stakeholder de dominio	Expresar necesidades, validar significados y aclarar terminología.	Obligatorio.
Responsable de decisión / PO	Priorizar, confirmar alcance y aprobar criterios.	Obligatorio cuando el rol existe en el contexto.
Representante técnico / QA	Evaluar factibilidad, verificabilidad y testabilidad.	Recomendado.
Equipo investigador externo	Evaluar utilidad, claridad, carga y calidad de artefactos.	Externo al proceso operativo.

Tabla 5.4: Criterios de activación contextual en THUNDERS-Master.

Criterio de activación	Elemento activado	Evidencia esperada
Alta heterogeneidad de stakeholders	Subgrupos, rondas de participación o reformulación cruzada.	Registro de perspectivas y acuerdos integrados.
Requisitos críticos o regulados	Trazabilidad extendida y validación más formal.	Enlaces completos necesidad–requisito–criterio–decisión–evidencia.
Distribución geográfica o comunicación asíncrona	Canvas reforzado, glosario vivo y decision log más explícito.	Acuerdos visibles para quienes no asistieron.
Baja madurez en IR	Plantillas de una página, checklist de calidad y preguntas guía.	Requisitos con estructura mínima y criterios observables.
Terminología de dominio ambigua	Glosario vivo y responsable de resolución semántica.	Términos críticos definidos o marcados como pendientes.
Discrepancias no resueltas	Sesión adicional o proceso de escalamiento.	Discrepancia asignada con responsable, fecha y criterio de cierre.

5.10. Estado de madurez y evolución del proceso

THUNDERS-Master se asume como una especificación de trabajo en evolución. Su construcción y refinamiento siguen la lógica iterativa e incremental con que se construyó THUNDERS, sometido a sucesivas validaciones que corregían y enriquecían cada versión [7, 8]. La Tabla 5.5 describe la ruta de evolución prevista: la versión v1.0 (la especificada en este capítulo) alimenta un ciclo de validación experta y pilotaje en contexto agrícola, del cual se derivarán versiones posteriores. Cada iteración debe registrar qué cambió, por qué cambió y con qué evidencia, de modo que la trazabilidad entre versiones se conserve.

Tabla 5.5: Ruta de evolución prevista para THUNDERS-Master.

Versión	Entrada / fuente de cambios	Resultado esperado
v1.0	RSL, caracterización, análisis de THUNDERS y SME.	Especificación formal de trabajo (este capítulo).
v1.1	Taller de validación con expertos y usuarios clave (coherencia interna, pertinencia de actividades, claridad operativa).	Versión refinada lista para pilotar, con ajustes documentados.
v2.0	Pilotaje en estudio de caso agrícola y ejecución del plan GQM (utilidad, facilidad de uso, carga cognitiva, calidad de artefactos).	Versión ajustada con lecciones aprendidas y evidencia empírica.
Futuras	Réplicas en otros dominios y contextos.	Posible línea de procesos (SPrL) con variantes especializadas.

Estado del proceso

Los resultados de las iteraciones v1.1 y v2.0 se reportarán en el Capítulo 6. Mientras esas validaciones no se ejecuten, las afirmaciones sobre utilidad y facilidad de uso de THUNDERS-Master deben entenderse como hipótesis de diseño, no como hallazgos confirmados.

Capítulo 6

Evaluación de THUNDERS-Master

6.1. Generalidades

La evaluación se separa del uso operativo del proceso: durante una sesión real no se incorpora un rol investigador que evalúe desempeño individual ni se aplican instrumentos extensos. La evaluación pertenece al proyecto de investigación y se orienta a validar la coherencia, utilidad, facilidad de uso y capacidad del proceso para producir evidencia de entendimiento compartido.

6.2. Plan de evaluación basado en GQM

El plan inicial se estructura mediante Goal-Question-Metric (GQM) [16]. El objetivo es evaluar si THUNDERS-Master ayuda a construir, monitorear y validar entendimiento compartido durante la elicitación, especificación y validación. A partir de este objetivo se formulan preguntas sobre reducción de ambigüedad, trazabilidad, discrepancias, calidad de criterios, utilidad percibida, facilidad de aplicación y carga cognitiva [74]. Las métricas se obtendrán mediante revisión de artefactos, checklist de calidad, decision log, encuestas breves —la escala SUS [75] puede servir como referencia de usabilidad—, entrevistas y observación del piloto. La Tabla 6.1 presenta la propuesta inicial.

6.3. Estrategia de evaluación

La evaluación se ejecuta en dos instancias complementarias, alineadas con las fases de la investigación (Sección 1.6) y con la ruta de versionado del proceso (Tabla 5.5). La primera es un *taller de validación con expertos y usuarios clave*, orientado a verificar la coherencia

Tabla 6.1: Plan de evaluación propuesto para THUNDERS-Master (GQM).

Objetivo	Pregunta / indicador	Instrumento
Evaluar utilidad	¿El proceso ayuda a construir entendimiento compartido? Malentendidos detectados, acuerdos logrados.	Cuestionario y revisión de artefactos.
Evaluar facilidad de uso	¿Se aplica sin esfuerzo excesivo? Claridad de pasos, tiempo de aplicación.	Cuestionario de usabilidad (SUS).
Evaluar carga cognitiva	¿Reduce el esfuerzo mental frente a THUNDERS completo?	NASA-TLX.
Evaluar calidad de artefactos	¿Los requisitos son comprensibles, trazables y validables?	Checklist de calidad de requisitos.
Evaluar conservación del propósito	¿Mantiene construcción, monitoreo y validación del entendimiento?	Matriz de trazabilidad entre THUNDERS y THUNDERS- Master.

interna de THUNDERS-Master, la pertinencia de sus actividades, roles e instrumentos, y la claridad operativa de sus pasos; su retroalimentación produce la versión a pilotar (v1.1) y permite revisar la alineación entre el proceso y el plan de medición GQM. La segunda es un *estudio de caso* que aplica el proceso en un contexto real (agrícola) y ejecuta el plan GQM para obtener evidencia empírica (v2.0). Esta separación evita confundir la ejecución operativa del proceso con su evaluación como investigación.

6.4. Diseño del estudio de caso en contexto agrícola

El pilotaje se realizará en un contexto agrícola por las razones expuestas en la Sección 1.4.4: diversidad técnica, disciplinar y comunicativa entre productores, asesores, técnicos, investigadores y diseñadores, que incrementa el riesgo de malentendidos y hace especialmente pertinente un proceso de alineación semántica [12, 14]. Tras seleccionar y formalizar el caso, la implementación se organiza en sesiones que corresponden a las tres actividades del alcance: *(i)* elicitación, con sesiones de levantamiento para capturar necesidades, restricciones y criterios de aceptación; *(ii)* especificación, estructurando lo capturado en artefactos acordados (por ejemplo, historias de usuario o especificación de requisitos) mediante las guías y plantillas de THUNDERS-Master; y *(iii)* validación, con sesiones de revisión y acuerdo que culminan en una versión acordada del conjunto de requisitos.

Durante la evaluación se ejecuta el plan GQM con los instrumentos definidos. El análisis combina estadística descriptiva para las métricas de percepción y análisis temático para

la información cualitativa de entrevistas y observaciones, con el fin de identificar patrones de entendimiento compartido y puntos de fricción. Con base en este análisis se elabora un registro de lecciones aprendidas y se definen los ajustes a THUNDERS-Master.

6.5. Validación por expertos

☐ Sección pendiente

Reportar la ejecución del taller de validación con expertos y usuarios clave: perfil y número de participantes, instrumento (rúbrica de completitud, coherencia, claridad, aplicabilidad y alineación con IR), protocolo de aplicación, hallazgos y ajustes realizados a la especificación (v1.0 → v1.1).

6.6. Walkthrough metodológico

☐ Sección pendiente

Realizar un recorrido (walkthrough) paso a paso del proceso sobre un caso representativo para verificar coherencia interna, dependencias entre actividades y completitud de artefactos. Documentar inconsistencias y correcciones.

6.7. Resultados del pilotaje en contexto agrícola

☐ Sección pendiente

Presentar el desarrollo del estudio de caso: contexto y participantes, configuración del proceso (núcleo y elementos activados), ejecución de las sesiones de elicitación, especificación y validación, y recolección de evidencia según el plan GQM. Incluir las amenazas a la validez del estudio.

6.8. Análisis de resultados y lecciones aprendidas

☐ Sección pendiente

Presentar y discutir los resultados (utilidad, facilidad de uso, carga cognitiva, calidad de artefactos y conservación del propósito), compararlos con las expectativas del plan GQM, consolidar el registro de lecciones aprendidas y derivar la versión ajustada de THUNDERS-Master (v2.0).

Capítulo 7

Conclusiones y trabajo futuro

7.1. Generalidades

Este capítulo recoge las conclusiones del trabajo, sus contribuciones, limitaciones y las líneas de trabajo futuro.

7.2. Conclusiones

El entendimiento compartido es una condición crítica para la calidad de los requisitos, porque permite que stakeholders y equipos técnicos interpreten de forma coherente problemas, necesidades, restricciones y criterios de validación. La RSL muestra que las rupturas de significado aparecen en la identificación de stakeholders, el lenguaje del dominio, la captura de necesidades, la especificación, la trazabilidad y la validación. Sobre esa base, la revisión delimitó una brecha concreta (Sección 2.4.13): no existe una ruta integrada, ligera y trazable para construir, monitorear y validar el entendimiento compartido, lo que justifica adaptar THUNDERS en lugar de crear un proceso nuevo. A partir de esa evidencia y de la caracterización de las tres actividades, este trabajo definió THUNDERS-Master: un proceso ligero, derivado de THUNDERS mediante SME, para construir, monitorear y validar el entendimiento compartido en IR. La especificación formaliza requisitos del método, decisiones de adaptación, tres fases, once actividades operativas, roles simplificados, artefactos mínimos, criterios de activación contextual y un plan inicial de evaluación.

□ Sección pendiente

Completar las conclusiones con los resultados de la evaluación (Capítulo 6) una vez ejecutada la validación experta y el piloto.

7.3. Contribuciones

Las contribuciones del trabajo, alineadas con la Sección 1.6 y los aportes descritos en el Capítulo 1, son de tres tipos. En el ámbito *académico*, amplía la comprensión teórica y práctica del entendimiento compartido en IR, ofreciendo una definición operacional y una trazabilidad explícita entre THUNDERS y THUNDERS-Master. En el ámbito *práctico/industrial*, aporta un proceso formal pero liviano —con fases, actividades A1–A11, roles simplificados, artefactos mínimos y criterios de activación contextual— que busca reducir ambigüedades, retrabajo y errores de interpretación. En el ámbito *investigativo*, extiende la aplicación de THUNDERS al dominio de la IR y prepara la generación de evidencia empírica en un contexto agrícola.

□ Sección pendiente

Consolidar las contribuciones con la evidencia obtenida tras la validación experta y el pilotaje (Capítulo 6).

7.4. Limitaciones

La principal limitación es que esta versión sigue siendo una especificación formal de trabajo. Aunque está fundamentada en la RSL, la caracterización de IR, el análisis de THUNDERS y SME, todavía no debe presentarse como validada empíricamente.

□ Sección pendiente

Ampliar las limitaciones con las amenazas a la validez identificadas durante la evaluación.

7.5. Trabajo futuro

El trabajo futuro inmediato consiste en ejecutar la revisión experta, el walkthrough metodológico y un piloto o estudio de caso, analizando claridad, utilidad, facilidad de aplicación, carga percibida, calidad de artefactos y señales de entendimiento compartido. A mediano plazo se plantea construir un repositorio de *method chunks* THUNDERS-RE y explorar una línea de procesos (SPrL) que permita derivar variantes para otros dominios (educación, salud, contextos empresariales).

7.6. Productos de investigación

☐ Sección pendiente

Listar los productos derivados del trabajo: artículos enviados o publicados, ponencias y demás actividades de investigación.

Bibliografía

- [1] ISO/IEC/IEEE. ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering. Technical report, International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers, Geneva, Switzerland, 2018.
- [2] Bashar Nuseibeh and Steve Easterbrook. Requirements engineering: A roadmap. In *Proceedings of the Conference on the Future of Software Engineering*, pages 35–46, 2000.
- [3] Robert C. Fuller and Philippe Kruchten. Blurring boundaries: Toward the collective empathic understanding of product requirements. *Information and Software Technology*, 140:106670, 2021.
- [4] E. A. Christiane Bittner and Jan Marco Leimeister. Creating shared understanding in heterogeneous work groups: Why it matters and how to achieve it. *Journal of Management Information Systems*, 31(1):111–144, 2014.
- [5] Mehdi Rajabi Asadabadi, Morteza Saberi, Ofer Zwikael, and Elizabeth Chang. Ambiguous requirements: A semi-automated approach to identify and clarify ambiguity in large-scale projects. *Computers and Industrial Engineering*, 149:106828, 2020.
- [6] Davide Fucci, Emil Alégroth, and Tobias Axelsson. When traceability goes awry: An industrial experience report. *Journal of Systems and Software*, 192:111389, 2022.
- [7] Vanessa Agredo-Delgado. *A process to improve computer-supported collaborative work through the construction, monitoring and assistance of shared understanding in problem-solving activities*. PhD thesis, Universidad del Cauca, Popayán, Colombia, 2022.
- [8] Vanessa Agredo-Delgado, Pablo H. Ruiz, Cesar A. Collazos, and Fernando Moreira. An exploratory study on the validation of thunders: A process to achieve shared understanding in problem-solving activities. *Informatics*, 9(2):39, 2022.

- [9] Ivo Mattmann et al. The inscrutable jungle of quality criteria – how to formulate requirements for a successful product development. *Procedia CIRP*, 36:153–158, 2015.
- [10] Tawfeeq Alsanoosy, Maria Spichkova, and James Harland. The influence of power distance on requirements engineering activities. *Procedia Computer Science*, 159:2394–2403, 2019.
- [11] Roberto Grande et al. Is it worth adopting devops practices in global software engineering? possible challenges and benefits. *Computer Standards & Interfaces*, 87:103767, 2024.
- [12] Jonathan M. Getson et al. Understanding scientists’ communication challenges at the intersection of climate and agriculture. *PLOS ONE*, 17(8):e0269927, 2022.
- [13] Fabián López, Tiago Da Silva, José A. Pino, and Alicia Martínez. Crowdre4df: A crowd-based requirements engineering method for developing digital farming solutions. In *IFIP Advances in Information and Communication Technology*, volume 632, pages 310–326, 2021.
- [14] Rohana Sulaiman, Andy Hall, T. S. V. Reddy, and K. Dorai. Icts and innovation systems in agricultural development: From knowledge sharing to knowledge co-creation. *Agricultural Systems*, 188:103026, 2021.
- [15] Kai Petersen, Robert Feldt, Shahid Mujtaba, and Michael Mattsson. Systematic mapping studies in software engineering. In *Proceedings of the 12th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, pages 68–77, 2008.
- [16] Victor R. Basili, Gianluigi Caldiera, and H. Dieter Rombach. The goal question metric approach. In *Encyclopedia of Software Engineering*. Wiley, 1994.
- [17] Samuel Lim, Aron Henriksson, and Jelena Zdravkovic. Data-driven requirements elicitation: A systematic literature review. *SN Computer Science*, 2:16, 2021.
- [18] Woubshet Behutiye, Pilar Rodríguez, Markku Oivo, Sanja Aaramaa, Jari Partanen, and Antonin Abhervé. Towards optimal quality requirement documentation in agile software development: A multiple case study. *Journal of Systems and Software*, 183:111112, 2022.
- [19] Tim Spijkman et al. Alignment and granularity of requirements and architecture in agile development: A functional perspective. *Information and Software Technology*, 133:106535, 2021.

- [20] Qiang Li and Fanjiang Zeng. Enhancing software architecture adaptability: A comprehensive evaluation method. *Symmetry*, 16(7):894, 2024.
- [21] Yi Wang, Chetan Arora, Xiao Liu, Thuong Hoang, Vasudha Malhotra, Ben Cheng, and John Grundy. Who uses personas in requirements engineering: The practitioners' perspective. *Information and Software Technology*, 178:107609, 2025.
- [22] Paul Ralph. The two paradigms of software development research. *Science of Computer Programming*, 156:68–89, 2018.
- [23] Kim Dikert, Maria Paasivaara, and Casper Lassenius. Challenges and success factors for large-scale agile transformations: A systematic literature review. *Journal of Systems and Software*, 119:87–108, 2016.
- [24] Mohsin Irshad, Ricardo Britto, and Kai Petersen. Adapting behavior driven development (bdd) for large-scale software systems. *Journal of Systems and Software*, 177:110944, 2021.
- [25] Fabio Calefato, Filippo Lanubile, Tayana Conte, and Rafael Prikladnicki. Assessing the impact of real-time machine translation on multilingual meetings in global software projects. *Empirical Software Engineering*, 21(3):1002–1034, 2015.
- [26] Rashidah Kasauli, Eric Knauss, Jennifer Horkoff, Grischa Liebel, and Francisco Gomes de Oliveira Neto. Requirements engineering challenges and practices in large-scale agile system development. *Journal of Systems and Software*, 172:110851, 2021.
- [27] Nagadivya Balasubramaniam, Marjo Kauppinen, Antti Rannisto, Kari Hiekkanen, and Sari Kujala. Transparency and explainability of ai systems: From ethical guidelines to requirements. *Information and Software Technology*, 159:107197, 2023.
- [28] Dirk Burkhardt, Kawa Nazemi, Silvana Tomic, and Egils Ginters. Best-practice piloting of integrated social media analysis solution for e-participation in cities. *Procedia Computer Science*, 77:11–21, 2015.
- [29] Fahim Muhammad Khan, Javed Ali Khan, Muhammad Assam, Ahmed S. Almasoud, Abdelzahir Abdelmaboud, and Manar Ahmed Mohammed Hamza. A comparative systematic analysis of stakeholder's identification methods in requirements elicitation. *IEEE Access*, 10:30982–31011, 2022.
- [30] Claude Y. Laporte, Guillaume Verret, and Mirna Muñoz. A software project that partially failed: A small organization that ignored the management and technical practices of software standards. *Computer*, 56(5):138–144, 2023.

- [31] Habib Ullah Khan, Mahmood Niazi, Mohamed El-Attar, Naveed Ikram, Siffat Ullah Khan, and Asif Qumer Gill. Empirical investigation of critical requirements engineering practices for global software development. *IEEE Access*, 9:93593–93613, 2021.
- [32] Muhammad Azeem Akbar, Wishal Naveed, Sajjad Mahmood, Saima Rafi, Ahmed Alsanad, Abeer Abdul-Aziz Alsanad, Abdu Gumaei, and Abdulrahman Alothaim. Prioritization of global software requirements’ engineering barriers: An analytical hierarchy process. *IET Software*, 15(4):277–291, 2021.
- [33] Cristina Ribeiro and Daniel Berry. The prevalence and severity of persistent ambiguity in software requirements specifications: Is a special effort needed to find them? *Science of Computer Programming*, 195:102472, 2020.
- [34] Ezeldin Sherif, Waleed Helmy, and Galal Hassan Galal-Edeen. Proposed framework to manage non-functional requirements in agile. *IEEE Access*, 11:53995–54005, 2023.
- [35] Leandro Antonelli, Gustavo Rossi, and Alejandro Oliveros. A collaborative approach to describe the domain language through the language extended lexicon. *The Journal of Object Technology*, 15(3):3:1, 2016.
- [36] Masami Hayashi, Kaori Hayashi, Yohei Sonobe, Hiroyuki Sato, and Hironori Takeuchi. Specification description analysis method for requirement modeling. *Procedia Computer Science*, 246:189–198, 2024.
- [37] Aline Kunz, Sabrina Pohlmann, Oliver Heinze, Antje Brandner, Christina Reiß, Martina Kamradt, Joachim Szecsenyi, and Dominik Ose. Strengthening interprofessional requirements engineering through action sheets: A pilot study. *JMIR Human Factors*, 3(2):e25, 2016.
- [38] Emanuele Gentili and Davide Falessi. Practitioners’ perceptions on requirements smells. *Information and Software Technology*, 187:107823, 2025.
- [39] Xuan Xiao, Aron Lindberg, Sean Hansen, and Kalle Lyytinen. “computing” requirements for open source software: A distributed cognitive approach. *Journal of the Association for Information Systems*, pages 1217–1252, 2018.
- [40] Hugo Ferreira Martins, Antônio Carvalho de Oliveira Junior, Edna Dias Canedo, Ricardo Ajax Dias Kosloski, Roberto Ávila Paldês, and Edgard Costa Oliveira. Design thinking: Challenges for software requirements elicitation. *Information*, 10(12):371, 2019.

- [41] Abhimanyu Gupta, Geert Poels, and Palash Bera. Using conceptual models in agile software development: A possible solution to requirements engineering challenges in agile projects. *IEEE Access*, 10:119745–119766, 2022.
- [42] Carla Pacheco, Ivan Garcia, Jose A. Calvo-Manzano, and Magdalena Reyes. Measuring and improving software requirements elicitation in a small-sized software organization: a lightweight implementation of iso/iec/ieee 15939:2017 – systems and software engineering – measurement process. *Requirements Engineering*, 28(2):257–281, 2022.
- [43] Samuel A. Fricker, Kurt Schneider, Farnaz Fotrousi, and Christoph Thuemmler. Workshop videos for requirements communication. *Requirements Engineering*, 21(4):521–552, 2015.
- [44] Laura Okpara, Colin Werner, Adam Murray, and Daniela Damian. The role of informal communication in building shared understanding of non-functional requirements in remote continuous software engineering. *Requirements Engineering*, 28(4):595–617, 2023.
- [45] Marné De Vries and Petra Opperman. Improving active participation during enterprise operations modeling with an extended story-card-method and participative modeling software. *Software and Systems Modeling*, 22(4):1341–1368, 2023.
- [46] Franklin Parrales-Bravo, Rosangela Caicedo-Quiroz, Julio Barzola-Monteses, Leonel Vasquez-Cevallos, María Isabel Galarza-Soledispa, and Manuel Reyes-Wagnio. Sure: Structure for unambiguous requirement expression in natural language. *Electronics*, 13(11):2206, 2024.
- [47] Nazakat Ali and Jang-Eui Hong. Value-oriented requirements: Eliciting domain requirements from social network services to evolve software product lines. *Applied Sciences*, 9(19):3944, 2019.
- [48] Oleksandr Kosenkov, Ehsan Zabardast, Davide Fucci, Daniel Mendez, and Michael Unterkalmsteiner. Privacy by design: Aligning gdpr and software engineering specifications with a requirements engineering approach. *Information and Software Technology*, 190:107946, 2026.
- [49] ISO/IEC/IEEE. ISO/IEC/IEEE 15288:2015 – Systems and software engineering – System life cycle processes. Technical report, International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers, 2015.

- [50] ISO/IEC/IEEE. ISO/IEC/IEEE 12207:2017 – Systems and software engineering – Software life cycle processes. Technical report, International Organization for Standardization, International Electrotechnical Commission, and Institute of Electrical and Electronics Engineers, 2017.
- [51] ISO/IEC. ISO/IEC 25010:2023 – Systems and software engineering – SQuaRE – Product quality model. Technical report, International Organization for Standardization and International Electrotechnical Commission, 2023.
- [52] ISO. ISO 9241-11:2018 – Ergonomics of human-system interaction – Part 11: Usability: Definitions and concepts. Technical report, International Organization for Standardization, 2018.
- [53] ISO/IEC. ISO/IEC 25063:2014 – Systems and software engineering – Systems and software product Quality Requirements and Evaluation (SQuaRE) – Common Industry Format (CIF) for usability: Context of use description. Technical report, International Organization for Standardization and International Electrotechnical Commission, 2014.
- [54] ISO/IEC. ISO/IEC 25064:2013 – Systems and software engineering – Software product Quality Requirements and Evaluation (SQuaRE) – Common Industry Format (CIF) for usability: User needs report. Technical report, International Organization for Standardization and International Electrotechnical Commission, 2013.
- [55] Faiza Akram, Tanzila Ahmad, and Mehwish Sadiq. Recommendation systems-based software requirements elicitation process – a systematic literature review. *Journal of Engineering and Applied Science*, 71:29, 2024.
- [56] International Requirements Engineering Board (IREB). CPRE Foundation – Start Working Effectively in RE. <https://cpre.ireb.org/en/concept/foundationlevel>, 2026. Accessed: Apr. 23, 2026.
- [57] Daniela E. Damian and James Chisan. An empirical study of the complex relationships between requirements engineering processes and other processes that lead to payoffs in productivity, quality, and risk management. *IEEE Transactions on Software Engineering*, 32(7):433–453, 2006.
- [58] Gunnar Brataas, Antonio Martini, Geir K. Hanssen, and Georg Ræder. Agile elicitation of scalability requirements for open systems: A case study. *Journal of Systems and Software*, 182:111064, 2021.

- [59] Barbara Kitchenham and Stuart Charters. Guidelines for performing systematic literature reviews in software engineering. Technical Report EBSE-2007-01, Keele University and Durham University, 2007.
- [60] John Sweller. Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2):257–285, 1988.
- [61] Ton de Jong. Cognitive load theory, educational research, and instructional design: Some food for thought. *Instructional Science*, 38:105–134, 2010.
- [62] Brian Henderson-Sellers and Jolita Ralyté. Situational method engineering: State-of-the-art review. *Journal of Universal Computer Science*, 16(3):424–478, 2010.
- [63] Sjaak Brinkkemper. Method engineering: Engineering of information systems development methods and tools. *Information and Software Technology*, 38(4):275–280, 1996.
- [64] Jolita Ralyté and Colette Rolland. An assembly process model for method engineering. In *Advanced Information Systems Engineering (CAiSE)*, pages 267–283, 2001.
- [65] Brian Henderson-Sellers, Jolita Ralyté, Pär J. Ågerfalk, and Matti Rossi. *Situational Method Engineering*. Springer, Berlin, 2014.
- [66] A. Frank Harmsen and Sjaak Brinkkemper. Description and manipulation of method fragments for situational method assembly. In *Proceedings of the 5th International Workshop on Experience with the Management of Software Projects*, 1995.
- [67] Isabelle Mirbel and Jolita Ralyté. Situational method engineering: Combining assembly-based and roadmap-driven approaches. *Requirements Engineering*, 11(1):58–78, 2006.
- [68] Peng Xu and Balasubramaniam Ramesh. Software process tailoring: An empirical investigation. *Journal of Management Information Systems*, 24(2):293–328, 2007.
- [69] Mark P. Ginsberg and Linda Quinn. Process tailoring and the software capability maturity model. Technical Report CMU/SEI-94-TR-024, Software Engineering Institute, Carnegie Mellon University, 1995.
- [70] Georg Kalus and Marco Kuhrmann. Criteria for software process tailoring: A systematic review. In *Proceedings of the International Conference on Software and System Process (ICSSP)*, pages 171–180, 2013.

- [71] Marco Kuhrmann, Thomas Ternité, Jürgen Friedrich, Andreas Rausch, and Manfred Broy. Flexible software process lines in practice: A metamodel-based approach to effectively construct and manage families of software process models. *Journal of Systems and Software*, 121:49–71, 2016.
- [72] Amadeu Silveira Campanelli and Fernando Silva Parreiras. Agile methods tailoring: A systematic literature review. *Journal of Systems and Software*, 110:85–100, 2015.
- [73] Philippe Kruchten. Contextualizing agile software development. *Journal of Software: Evolution and Process*, 25(4):351–361, 2013.
- [74] Sandra G. Hart and Lowell E. Staveland. Development of nasa-tlx (task load index): Results of empirical and theoretical research. In *Human Mental Workload*, volume 52 of *Advances in Psychology*, pages 139–183. North-Holland, 1988.
- [75] John Brooke. Sus: A quick and dirty usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor and Francis, 1996.